

Online Data Storage

**JSON • REST API • HTTP •
PHP/MySQL CRUD • Image
Upload**

Eng | Ruqaih Hussein Salman

2027-2028

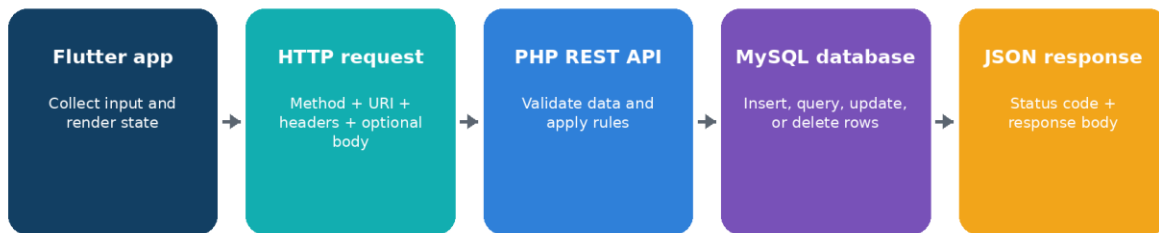
Lecture Overview and Learning Outcomes

- Explain the boundary Flutter client → HTTP request → PHP API → MySQL → JSON response.
- Recognize JSON objects, arrays, primitive values, nested objects, and null.
- Decode JSON and map values to a simple Dart data model.
- Read the local ProductItem JSON asset and render it with ListView.builder.
- Use package:http for GET, POST, PUT/PATCH, and DELETE requests.
- Fetch RandomUser data with package:http and display it with FutureBuilder.
- Build the Inventory Product CRUD flow using Flutter, PHP, and MySQL.
- Upload an image using image_picker + multipart/form-data and store a server path in MySQL.

LEARNING FLOW

Concept → visual explanation → focused code → expected output → practice.

Online data flow



Visual map — Flutter client → HTTP request → PHP REST API → MySQL → JSON response.

What Each Layer Does

Boundary	Owns	Must not own
Flutter client	User input, UI state, request construction, JSON parsing	MySQL credentials or raw SQL
HTTP/API contract	Method, endpoint, headers/body, status and response shape	Undocumented behavior that the client must guess
PHP REST API	Validation, authorization, prepared SQL, stable JSON responses	Trusting client input or exposing raw database errors
MySQL	Persistent rows, keys, constraints and transactions	Direct mobile-app connections
Flutter response layer	Status validation, typed models, loading/data/empty/error UI	Updating visible state before the server confirms the action

MENTAL MODEL

The Flutter app communicates with the HTTP API. The API handles database operations and returns JSON that Flutter can display.

Table of Contents

Lecture topics and learning path

Part	Coverage	Page
1	API contract and request/response parts	4
2	JSON format and Dart data model	6
3	Local ProductItem JSON asset	8
4	REST API with package:http	14
5	HTTP methods and CRUD	20
6	Flutter + PHP + MySQL project	22
7	Image upload and server path	50
8	Errors, security, and testing	56
9	Guided practical lab	58
10	Knowledge check and answer key	59
11	References and final learning map	60

1. Application Programming Interface (API)

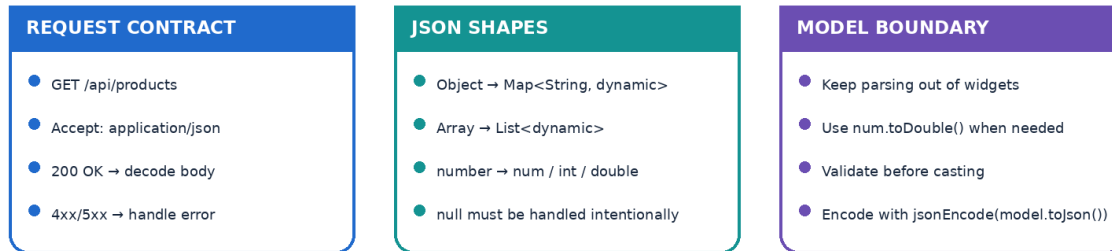
A request contract defines exactly how software systems communicate

PURPOSE Understand the six parts of an API call before writing Flutter code.	KEY API method • endpoint • headers • body • status • response	CHECKPOINT You can label every part of a request/response pair.
--	--	---

Part	Example	Meaning
Method	GET	Requested action
Endpoint	/api/products	Target resource
Headers	Content-Type: application/json	Metadata/authentication
Body	{"name":"Book"}	Data sent to server
Status	200 / 201 / 400 / 404 / 500	Outcome category
Response	JSON object or array	Returned data/error details

API Contract + JSON + Typed Dart Model

A compact learning map for the request/response contract and parsing boundary

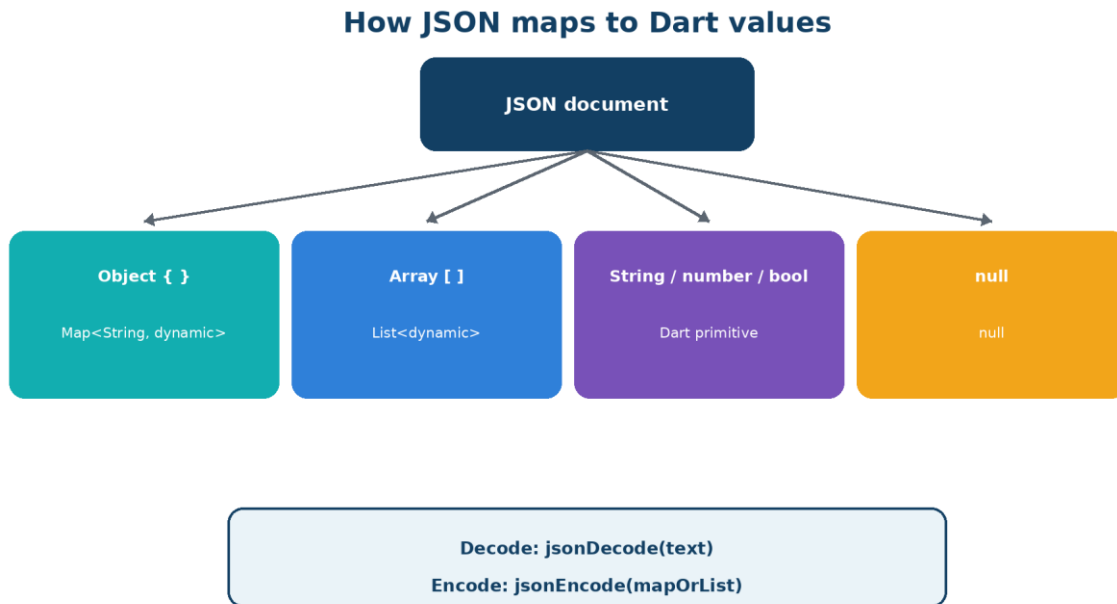


Checkpoint: decode only after validating the HTTP status code and the expected JSON shape.

Visual topic map: API contract, JSON conversion, and Dart data model.

2. JSON Format and Parsing

Understand the JSON data shape before writing Flutter code



Visual map: JSON object → Map, JSON array → List, primitives → Dart values.

JSON OBJECT AND JSON ARRAY

```
{
  "name": "Ruqaih",
  "age": 28,
  "active": true,
  "skills": ["Flutter", "Git"],
  "address": {"city": "Sana'a"},
  "photo": null
}

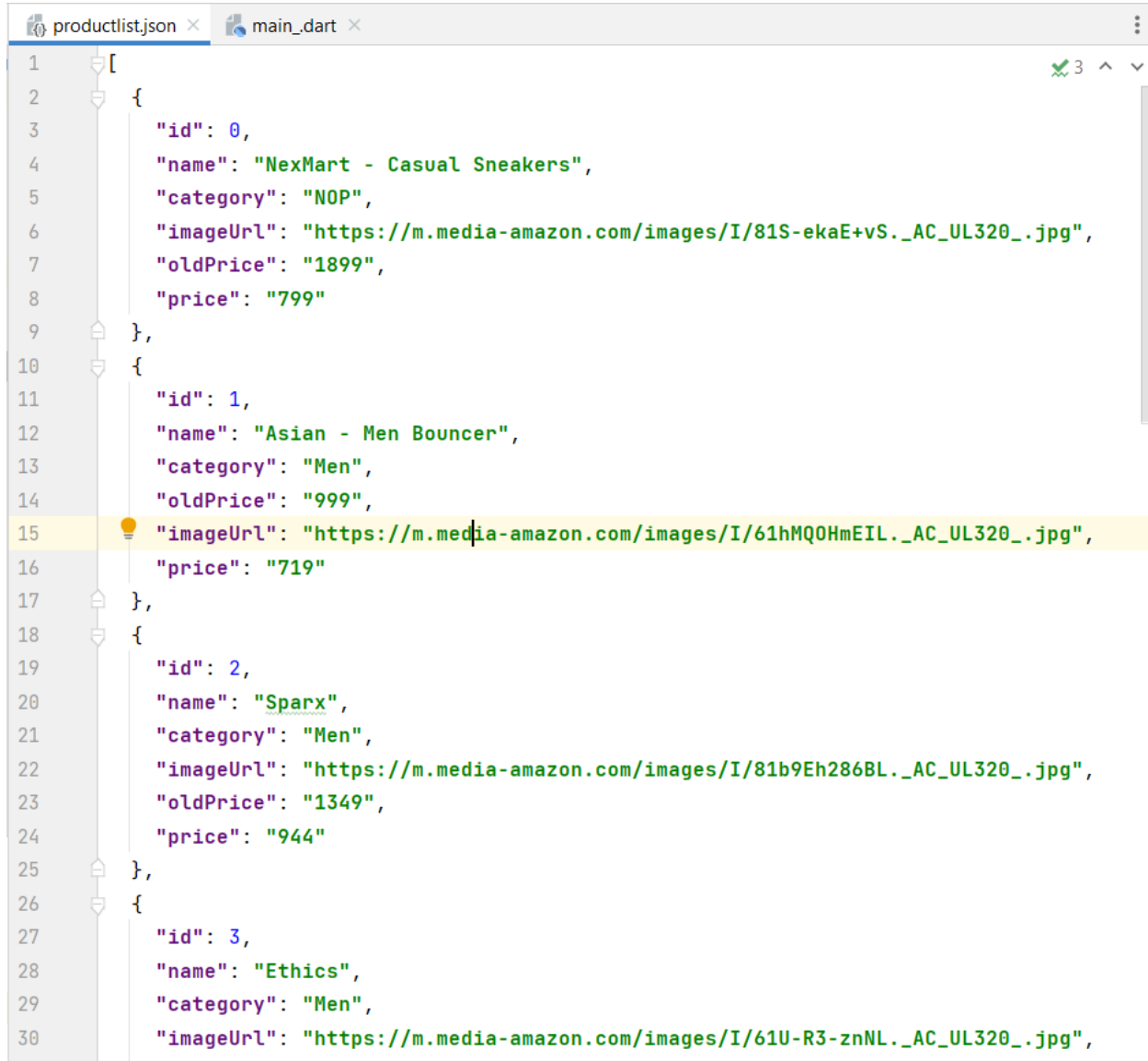
[
  {"id": 1, "name": "Alberto"},
  {"id": 2, "name": "Patrick"}
]
```

2.0 Original 2026 JSON Example

The original lecture begins with a real ProductItem JSON file. Keep this screenshot beside the redesigned JSON map so students can connect the concept to the actual classroom file.

READ IT IN THIS ORDER

Identify the outer array, then one object, then its keys and values. The same keys must appear in ProductDataModel.fromJson().



```

1  [
2    {
3      "id": 0,
4      "name": "NexMart - Casual Sneakers",
5      "category": "NOP",
6      "imageUrl": "https://m.media-amazon.com/images/I/81S-ekaE+vS._AC_UL320_.jpg",
7      "oldPrice": "1899",
8      "price": "799"
9    },
10   {
11     "id": 1,
12     "name": "Asian - Men Bouncer",
13     "category": "Men",
14     "oldPrice": "999",
15     "imageUrl": "https://m.media-amazon.com/images/I/61hMQOHmEIL._AC_UL320_.jpg",
16     "price": "719"
17   },
18   {
19     "id": 2,
20     "name": "Sparx",
21     "category": "Men",
22     "imageUrl": "https://m.media-amazon.com/images/I/81b9Eh286BL._AC_UL320_.jpg",
23     "oldPrice": "1349",
24     "price": "944"
25   },
26   {
27     "id": 3,
28     "name": "Ethics",
29     "category": "Men",
30     "imageUrl": "https://m.media-amazon.com/images/I/61U-R3-znNL._AC_UL320_.jpg",

```

Original 2026 screenshot - ProductItem JSON array used in the local asset example.

2.1 Create a Simple Data Model

PURPOSE Convert one JSON object into a Dart object.	KEY API Map<String, dynamic> → ProductDataModel.fromJson	CHECKPOINT Each JSON field is assigned to the matching variable.
---	---	--

PRODUCTDATAMODEL.DART

```

class ProductDataModel {
  int? id;
  String? name;
  String? category;
  String? imageURL;
  String? oldPrice;
  String? price;

  ProductDataModel({
    this.id,
    this.name,
    this.category,
    this.imageURL,
    this.oldPrice,
    this.price,
  });

  ProductDataModel.fromJson(Map<String, dynamic> json) {
    id = json['id'];
    name = json['name'];
    category = json['category'];
    imageURL = json['imageUr1'];
    oldPrice = json['oldPrice'];
    price = json['price'];
  }
}

```

DATA TYPE NOTE

In this ProductItem example, oldPrice and price are stored as strings, so the introductory model keeps them as String?.

3. Local JSON Asset — ProductItem Example

Load a JSON file from the Flutter project and display it with ListView.builder

PURPOSE Load JSON from the project before adding network requests.	KEY API pubspec assets → rootBundle.loadString → jsonDecode → ProductDataModel.fromJson	CHECKPOINT The shoe list is displayed from the local JSON file.
--	---	---

Step	What the student does	Why it matters
1	Create/copy productlist.json	Provides the JSON array.
2	Register the file in pubspec.yaml	Makes the asset available to Flutter.
3	Create ProductDataModel	Maps JSON fields to Dart variables.
4	Read with rootBundle	Loads the JSON text asynchronously.
5	Decode + map	Creates a list of ProductDataModel objects.
6	Display with FutureBuilder + ListView.builder	Shows the data on screen.

PROJECT STRUCTURE + PUBSPEC

```

project/
  jsonfile/
    productlist.json
  lib/
    main.dart
    ProductDataModel.dart

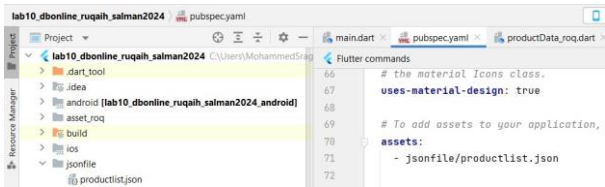
flutter:
  assets:
    - jsonfile/productlist.json

```

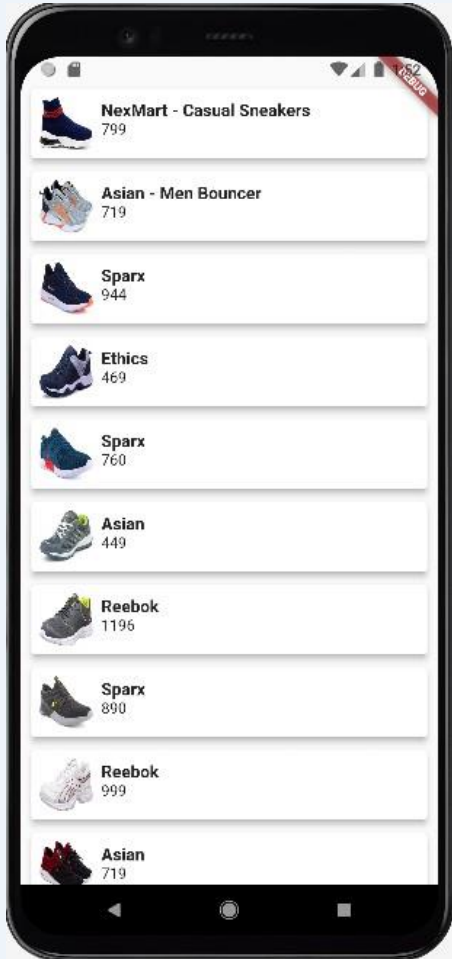
3.0 Original Project Setup and Output

These original screenshots show the exact classroom workflow: place the JSON file in assets, register it in pubspec.yaml, then run the list screen.

Android Studio project: assets/ProductItem.json



Original local JSON ListView output



Original 2026 practical sequence - add the asset, declare it, parse it, and display the product list.

3.1 Read and Parse the Asset

PURPOSE Read the local JSON file and decode its array.	KEY API <code>rootBundle.loadString + jsonDecode</code>	CHECKPOINT A <code>Future<List<ProductDataModel>></code> is ready for the UI.
--	---	---

READ JSON DATA

```
import 'dart:convert';
import 'package:flutter/services.dart' as rootBundle;

Future<List<ProductDataModel>> readJsonData() async {
  final jsonData = await rootBundle.rootBundle
    .loadString('jsonfile/productlist.json');

  final list = json.decode(jsonData) as List<dynamic>;

  return list
    .map((item) => ProductDataModel.fromJson(item))
    .toList();
}
```

3.2 Display Data with FutureBuilder and ListView.builder

PURPOSE Connect the JSON Future to the screen.	KEY API FutureBuilder + ListView.builder	CHECKPOINT Loading, error, and data states are visible.
--	--	---

Future state	What the student sees	What to verify
waiting	CircularProgressIndicator	The file is loading.
error	Error message	The JSON could not be read.
data	Product ListView	Each JSON item becomes one row.

FLUTTER UI CORE

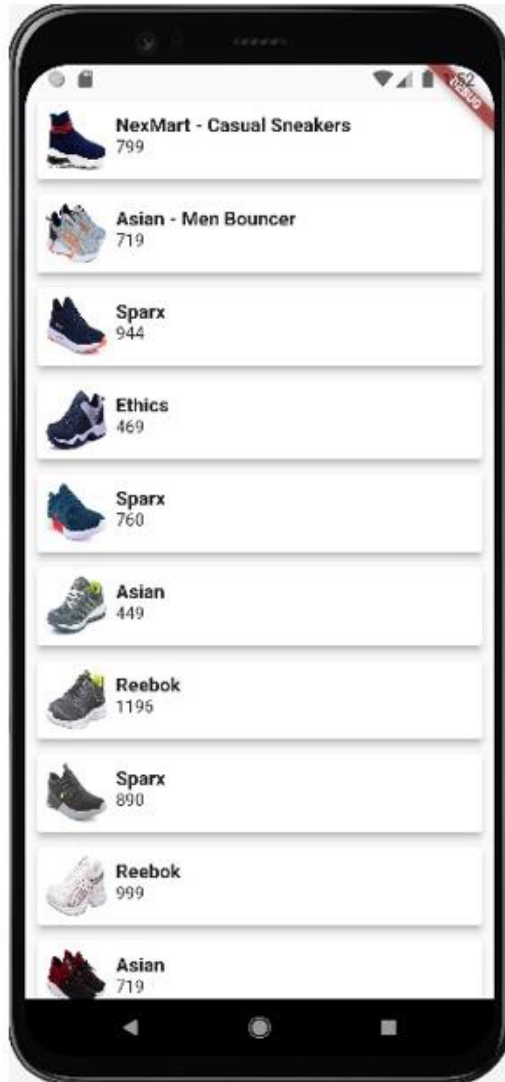
```
FutureBuilder<List<ProductDataModel>> (
  future: readJsonData(),
  builder: (context, data) {
    if (data.hasError) {
      return Center(child: Text('${data.error}'));
    }

    if (data.hasData) {
      final items = data.data!;

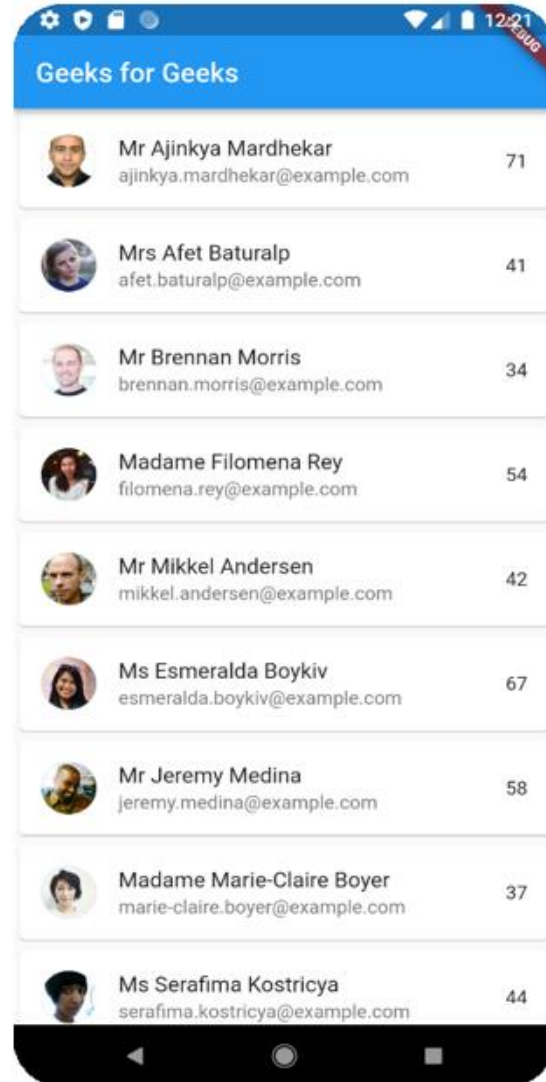
      return ListView.builder(
        itemCount: items.length,
        itemBuilder: (context, index) {
          return Card(
            child: ListTile(
              leading: Image.network(
                items[index].imageUrl.toString(),
                width: 50,
                height: 50,
              ),
              title: Text(items[index].name.toString()),
              subtitle: Text(items[index].category.toString()),
              trailing: Text(items[index].price.toString()),
            ),
          );
        },
      );
    }
  },
);

return const Center(
  child: CircularProgressIndicator(),
);
```

Local JSON Asset Output



REST API RandomUser Output



Application outputs — local ProductItem JSON (left) and RandomUser REST API (right).

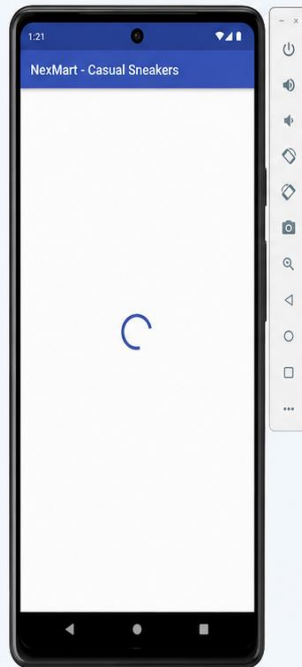
3.3 Visual Runtime Sequence — Loading → Data

See the loading state before the product list appears

PURPOSE Connect the Future state to what the student sees.	KEY API FutureBuilder waiting → data	CHECKPOINT The progress indicator changes into the product list.
--	--	--

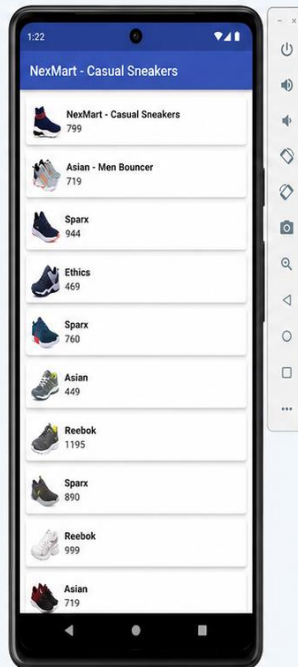
Lecture 10: Local JSON Assets & REST APIs in Android

1. Local JSON Asset - Loading



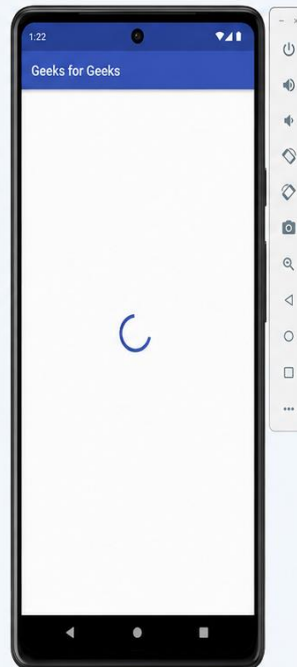
Pixel 6 API 34 (Android 14)

2. Local JSON Asset - Loaded



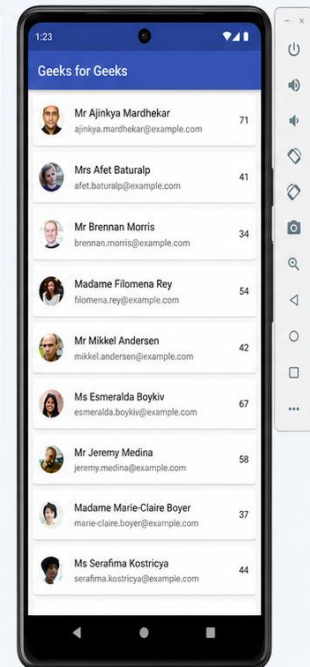
Pixel 6 API 34 (Android 14)

3. REST API RandomUser - Loading



Pixel 6 API 34 (Android 14)

4. REST API RandomUser - Loaded



Pixel 6 API 34 (Android 14)

Visual sequence — local JSON loading state and loaded product list.

4. REST API in Dart and Flutter

Fetch remote JSON from a REST API and display it in Flutter

PURPOSE Fetch JSON data from the internet.	KEY API package:http → http.get → jsonDecode	CHECKPOINT RandomUser data appears in the Flutter list.
--	--	---

SETUP

```
flutter pub add http

import 'package:http/http.dart' as http;

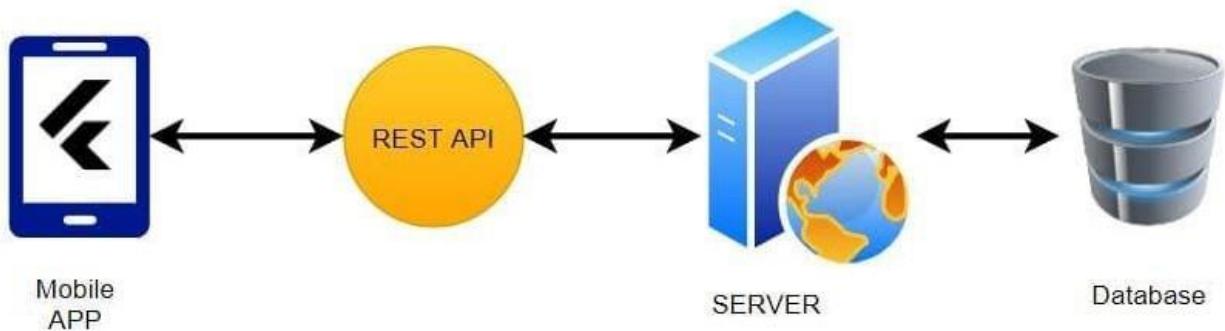
// Android network access:
// android/app/src/main/AndroidManifest.xml
<uses-permission android:name="android.permission.INTERNET" />
```

API URL

This example uses: <https://randomuser.me/api/?results=20>

4.0 Original REST API Walkthrough

The visual map is followed by the original REST API figures so the student can see both the architecture and the actual JSON/output used in class.

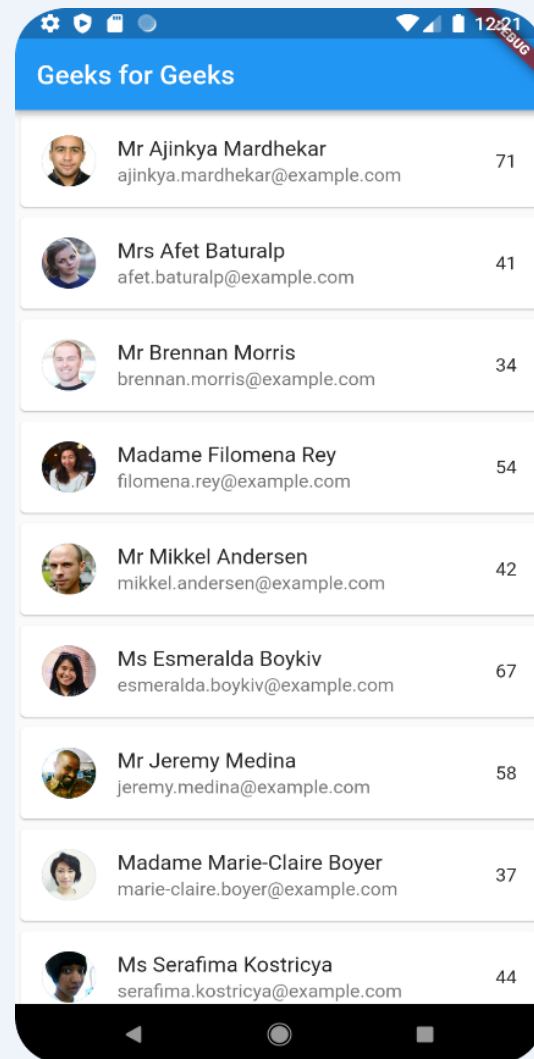


Original data path - Flutter mobile app to REST API server and database.

RandomUser JSON response in the browser

```
{
  "results": [
    {
      "gender": "female",
      "name": {
        "title": "Ms",
        "first": "Vaishnavi",
        "last": "Bharanya"
      },
      "location": {
        "street": {
          "number": 2169,
          "name": "College St"
        },
        "city": "Tadepalligudem",
        "state": "Uttar Pradesh",
        "country": "India",
        "postcode": 71453,
        "coordinates": {
          "latitude": 6.7354,
          "longitude": "12.8576"
        },
        "timezone": {
          "offset": "-12:00",
          "description": "Eniwetok, Kwajalein"
        }
      },
      "email": "vaishnavi.bharanya@example.com",
      "login": {
        "uuid": "f2b033ef-a8ce-45bf-85bd-89b97aedfeec",
        "username": "ticklishleopard658",
        "password": "punkrock",
        "salt": "HV6v50su",
        "md5": "88dbelid400434cd2c48c8b16e2b6ce5a",
        "sha1": "cc3e4481008a1ddc48d0180c0f813f396f138c56",
        "sha256": "7082799598554b229bf3ee03dcfa339fbef0e393acd2bd46e6f92b3e0a31c40"
      },
      "dob": {
        "date": "1984-01-15T03:52:17.606Z",
        "age": 39
      },
      "registered": {
        "date": "2014-02-26T06:01:56.715Z",
        "age": 9
      },
      "phone": "9989897293",
      "cell": "9154072251",
      "id": {
        "name": "UIDAI",
        "value": "393870074389"
      },
      "picture": {
        "large": "https://randomuser.me/api/portraits/women/47.jpg",
        "medium": "https://randomuser.me/api/portraits/med/women/47.jpg",
        "thumbnail": "https://randomuser.me/api/portraits/thumb/women/47.jpg"
      },
      "nat": "IN"
    },
    {
      "gender": "male",
      "name": {
        "title": "Mr",
        "first": "Hugo",
        "last": "White"
      }
    }
  ]
}
```

RandomUser data displayed in Flutter



Original Flutter + PHP REST API topic figure



Original 2026 REST API figures retained in the teaching sequence.

4.1 Fetch Users from the API

PURPOSE Send a GET request and return the results list.	KEY API <code>http.get</code> → <code>jsonDecode</code> → <code>['results']</code>	CHECKPOINT The function returns <code>Future<List<dynamic>></code> .
---	--	--

FETCHUSERS()

```
import 'dart:convert';
import 'package:http/http.dart' as http;

Future<List<dynamic>> fetchUsers() async {
  final response = await http.get(
    Uri.parse('https://randomuser.me/api/?results=20'),
  );

  if (response.statusCode == 200) {
    return jsonDecode(response.body)['results'];
  }

  throw Exception('Failed to load users');
}
```

4.2 Display API Data with FutureBuilder

PURPOSE Display the returned name, email, age, and avatar.	KEY API <code>FutureBuilder</code> + <code>ListView.builder</code>	CHECKPOINT The screen shows loading, error, or data.
--	--	--

State	UI	Teaching point
waiting	Spinner	The HTTP request is still running.
error	Error message	The request failed.
data	Avatar + name + email + age	The JSON result is displayed.

MYAPIWIDGET()

```

FutureBuilder<List<dynamic>> myApiWidget() {
  return FutureBuilder<List<dynamic>>({
    future: fetchUsers(),
    builder: (context, snapshot) {
      if (snapshot.hasError) {
        return Center(child: Text('${snapshot.error}'));
      }

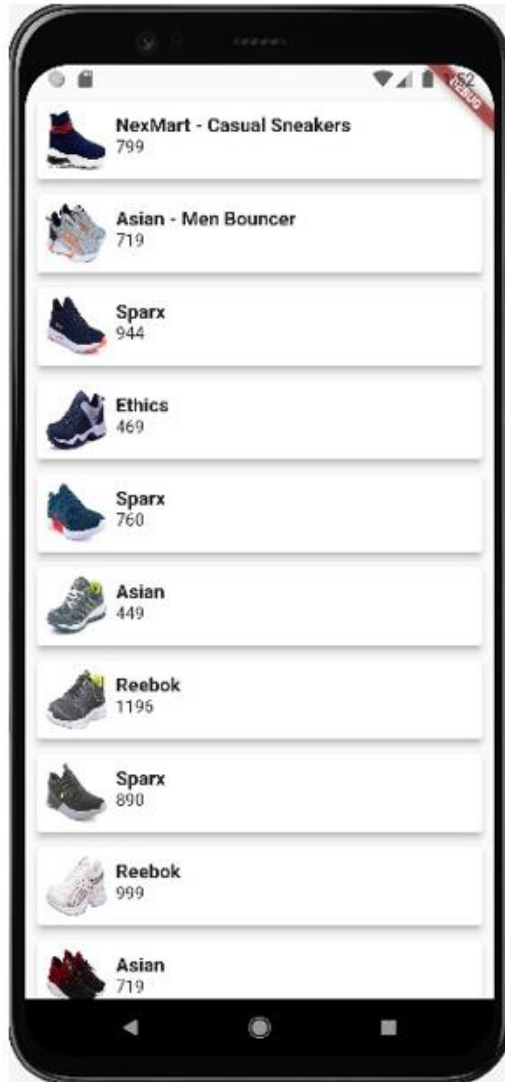
      if (snapshot.hasData) {
        return ListView.builder(
          itemCount: snapshot.data!.length,
          itemBuilder: (context, index) {
            final user = snapshot.data![index];

            return Card(
              child: ListTile(
                leading: CircleAvatar(
                  backgroundImage:
                    NetworkImage(user['picture']['large']),
                ),
                title: Text(
                  '${user['name']['title']} '
                  '${user['name']['first']} '
                  '${user['name']['last']}',
                ),
                subtitle: Text(user['email']),
                trailing: Text(user['dob']['age'].toString()),
              ),
            );
          },
        );
      }
    },
  );

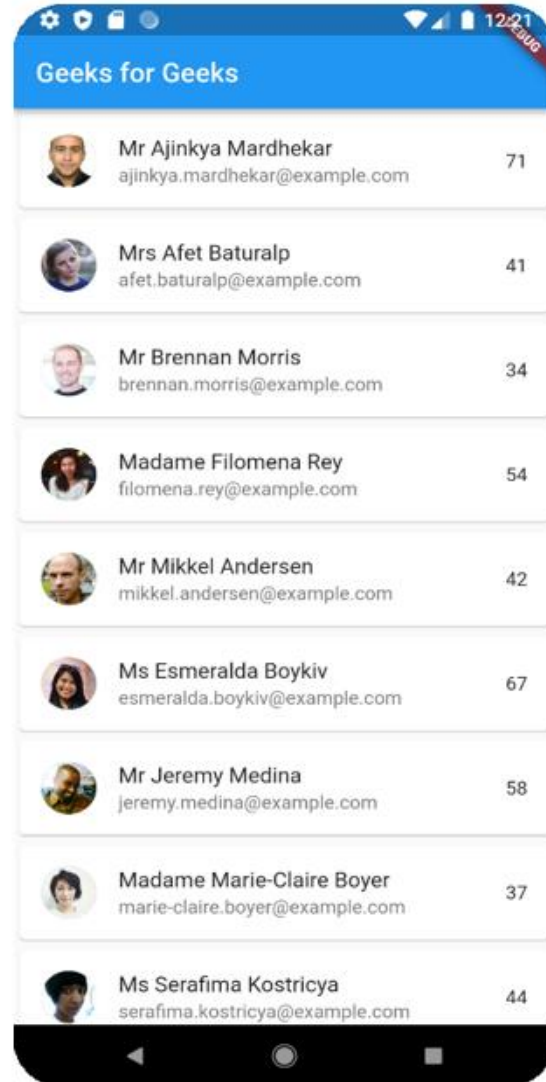
  return const Center(
    child: CircularProgressIndicator(),
  );
};
}

```

Local JSON Asset Output



REST API RandomUser Output

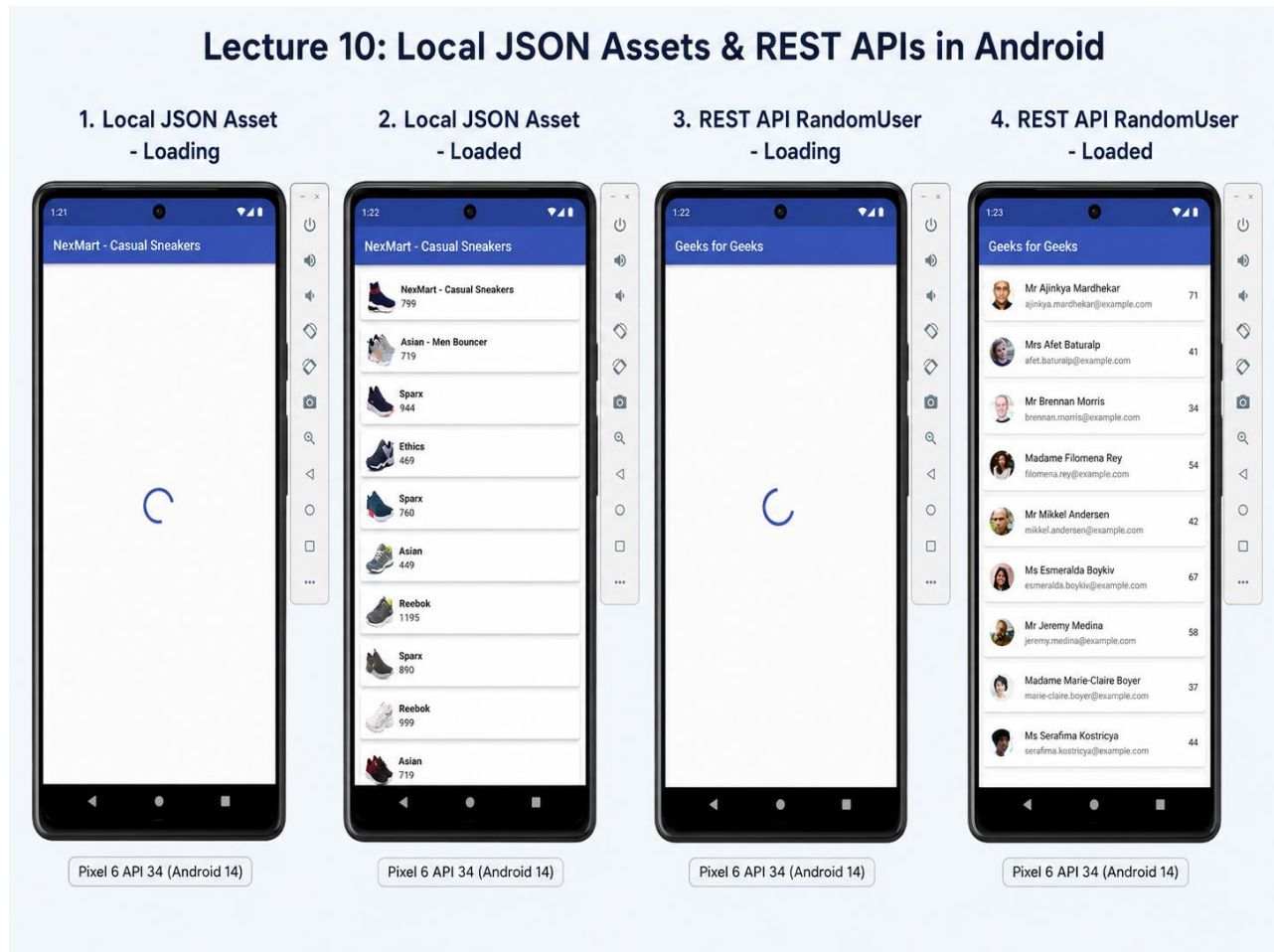


RandomUser API output — name, email, age, and avatar are displayed in the list.

4.3 Visual Runtime Sequence — Request → Data

PURPOSE See the request state before the list appears.	KEY API HTTP request → JSON → ListView	CHECKPOINT The spinner changes into the user list.
--	--	--

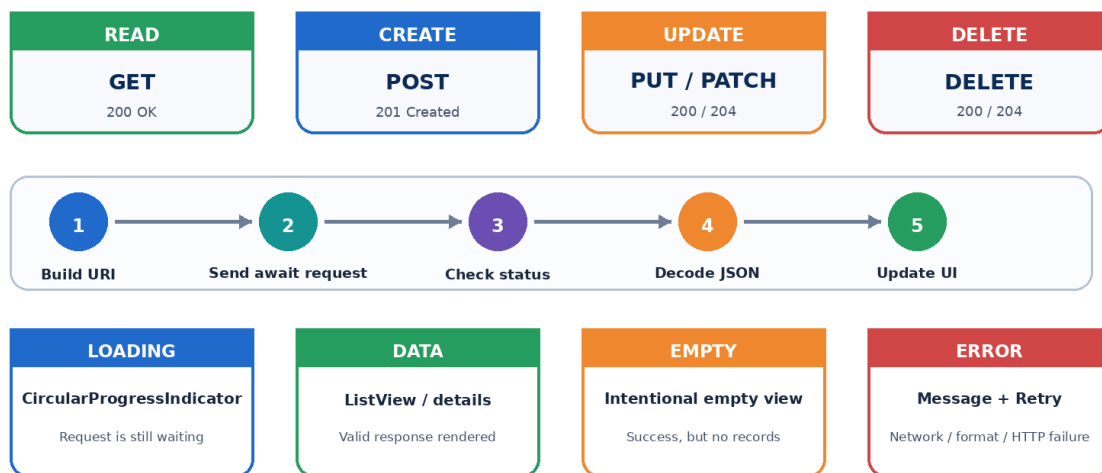
Lecture 10: Local JSON Assets & REST APIs in Android



Visual sequence — API request loading state and loaded RandomUser list.

HTTP CRUD + FutureBuilder UI States

Connect each user action to an HTTP method, status check, and visible UI state



Visual topic map: HTTP methods, FutureBuilder, and CRUD actions.

5. HTTP Methods and CRUD

5.1 REST Methods and CRUD

CRUD	HTTP	Typical success
Create	POST	201 Created
Read	GET	200 OK
Update all fields	PUT	200 OK / 204 No Content
Update selected fields	PATCH	200 OK / 204 No Content
Delete	DELETE	200 OK / 204 No Content

CONTRACT RULE

Status codes above are common conventions, not magic values. The client must follow the API contract it actually calls and validate the response before parsing JSON.

Connect each data action to the correct HTTP method

Action	Method	Request body	Typical check
Read	GET	Usually none	200
Create	POST	JSON	201
Replace/update	PUT	JSON	200 or 204
Partial update	PATCH	JSON subset	200 or 204
Delete	DELETE	Usually id in path/query	200 or 204

PURPOSE Recognize the main HTTP methods used by APIs.	KEY API http.get • http.post • http.put • http.delete	CHECKPOINT Choose the correct method for read, create, update, and delete.
---	---	--

SIMPLE HTTP METHOD EXAMPLES

```
// GET
final getResponse = await http.get(
  Uri.parse('$baseUrl/items'),
);

// POST
final postResponse = await http.post(
  Uri.parse('$baseUrl/items'),
  body: {'name': 'Mouse', 'price': '25'},
);

// PUT
final putResponse = await http.put(
  Uri.parse('$baseUrl/items/1'),
  body: {'name': 'Wireless Mouse'},
);

// DELETE
final deleteResponse = await http.delete(
  Uri.parse('$baseUrl/items/1'),
);
```

6. Flutter CRUD Application Using PHP REST API and MySQL

Build the Inventory Product app using Flutter → PHP API → MySQL

PURPOSE Connect the Flutter screens to PHP and MySQL.	KEY API Flutter client → PHP API → MySQL	CHECKPOINT Each screen reads or writes data through a PHP endpoint.
---	--	---

Layer	Responsibility	Must not do
Flutter client	Collect input, call endpoints, render states	Store DB credentials or run SQL.
PHP API	Validate, authorize, execute prepared SQL, return JSON	Trust client input or expose raw DB errors.
MySQL	Persist constrained relational rows	Accept direct mobile-app connections.

BOUNDARY RULE

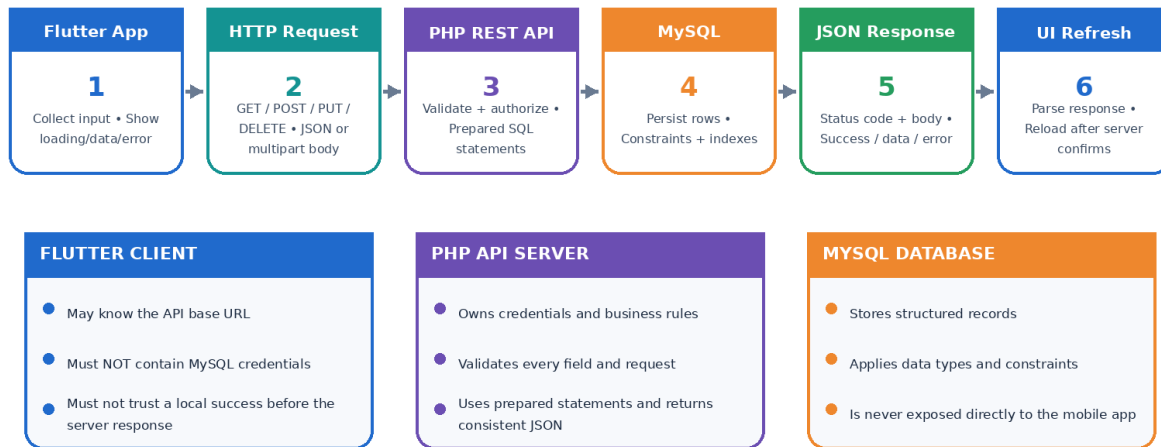
Flutter never connects directly to MySQL and never owns database credentials. PHP validates requests, performs SQL work, and returns JSON.

API Route Map

Route	Method	Used by
getdata.php	GET	List Data Inventory
adddata.php	POST	Add Data
editdata.php	POST	Edit Data
deleteData.php	POST	Detail / Delete
user_login.php	POST JSON	LoginForm_Roq
user_register.php	POST JSON	Sign Up

Flutter → PHP REST API → MySQL

The server boundary: responsibilities, data flow, and security rules



Security checkpoint: HTTPS + authentication + server-side validation + no raw database errors.

Visual topic map: Flutter → PHP REST API → MySQL CRUD boundary.

Flutter ↔ PHP CRUD Visual Connection Map

Pair the Flutter file with the PHP endpoint first; only then read the code.

THE FASTEST WAY TO STUDY THIS SECTION

For every CRUD operation, remember one pair of files. Flutter sends the request. PHP receives it, executes SQL, and returns JSON.

ACTION	FLUTTER	HTTP	PHP	SQL	RESULT
READ	listdata_roq.dart getData()	GET	getdata.php	SELECT	JSON list
CREATE	adddata_roq.dart addData()	POST	adddata.php	INSERT	new row
UPDATE	editdata_roq.dart editData()	POST	editdata.php	UPDATE	changed row
DELETE	detail_roq.dart deleteData()	POST	deleteData.php	DELETE	row removed

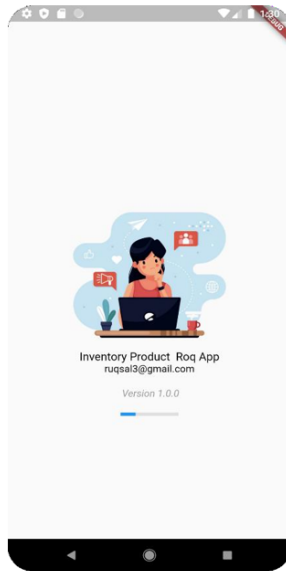
1	2	3	4	5	6
Screen action	Flutter function	HTTP request	PHP receives	SQL runs	UI refresh

ONE RULE

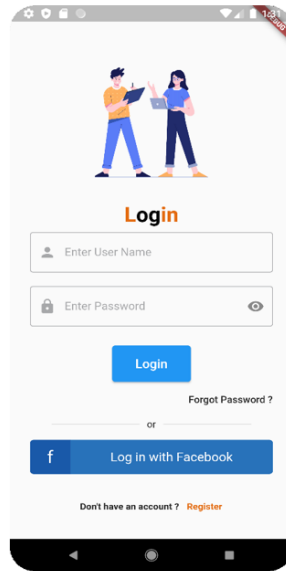
Do not memorize two separate programs. Read the Flutter function and its PHP file as one connected operation.

6.1 Application Screen Flow

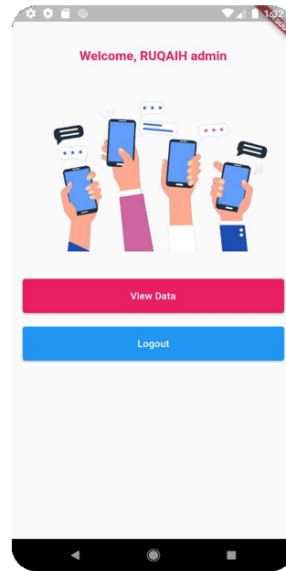
1 Splash



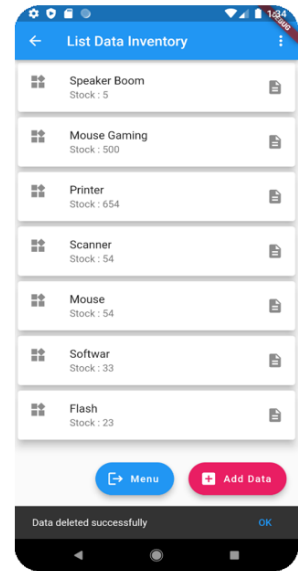
2 Login



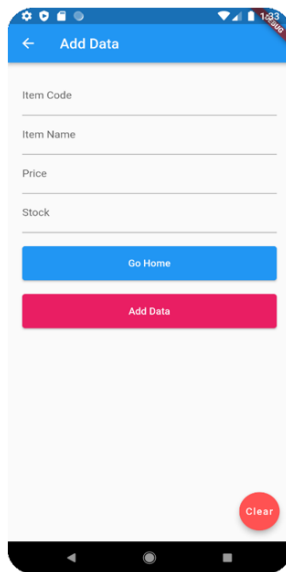
3 Home



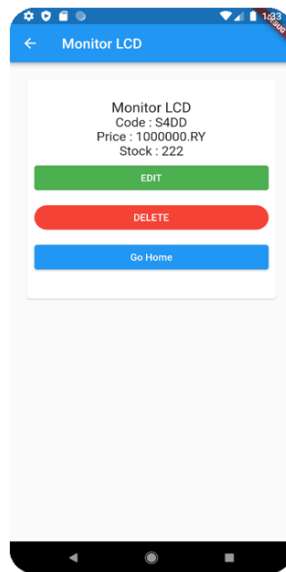
4 Read/List



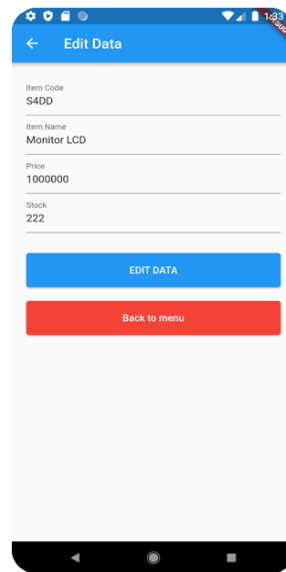
5 Create



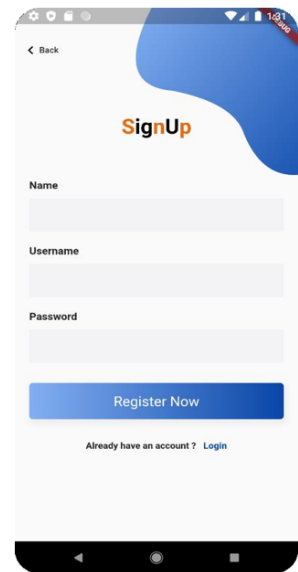
6 Detail/Delete



7 Update



8 Sign Up

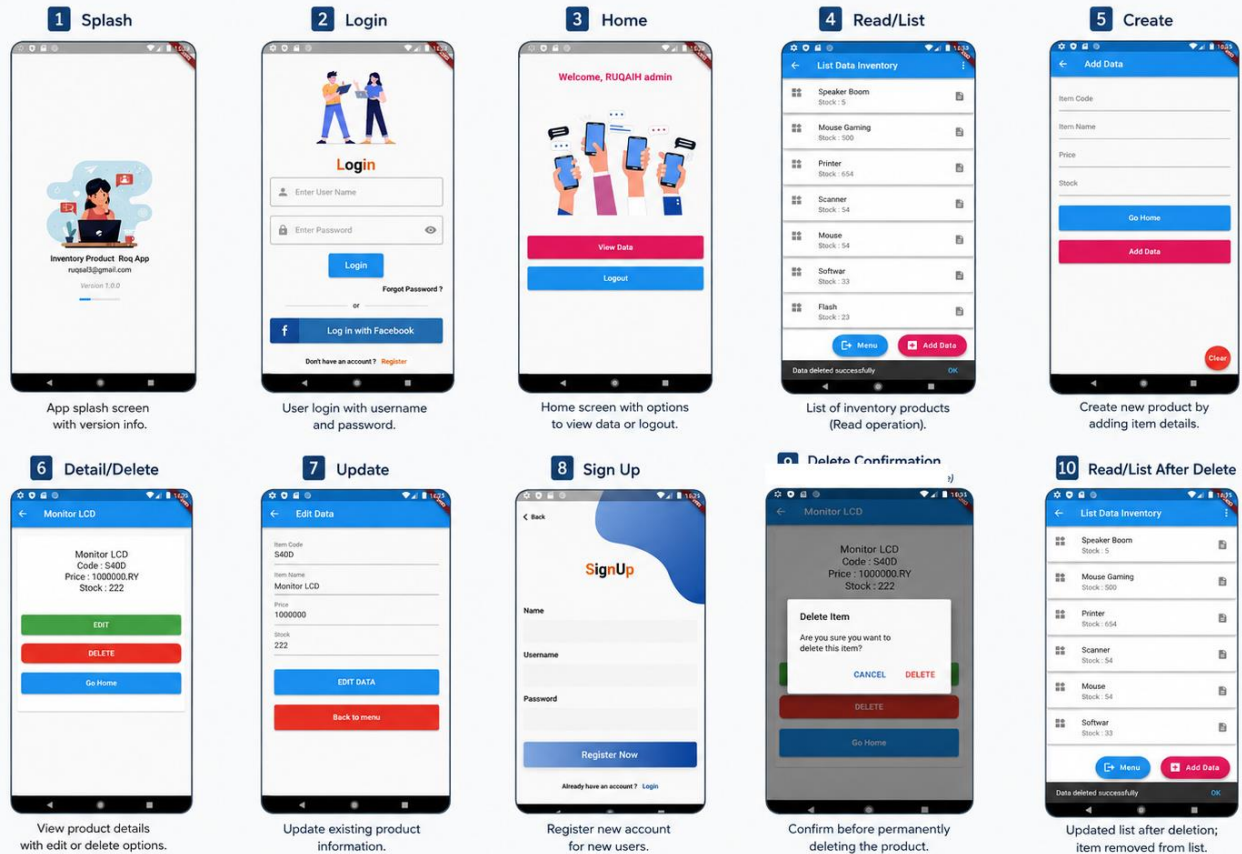


Inventory Product app screens: Splash, Login, Home, Read/List, Create, Detail/Delete, Update, and Sign Up.

6.1.1 CRUD Screen Story

PURPOSE Follow the complete screen flow from list to create, update, and delete.	KEY API Create → List → Detail → Update → List → Delete → List	CHECKPOINT Each action changes the data shown on the screen.
--	--	--

Lecture 10 | Inventory Product Roq App – CRUD Workflow



This workflow demonstrates CRUD operations: Create, Read (List), Update, Delete in the Inventory Product Roq App.

Visual sequence — Inventory Product CRUD screens from create through update and delete.

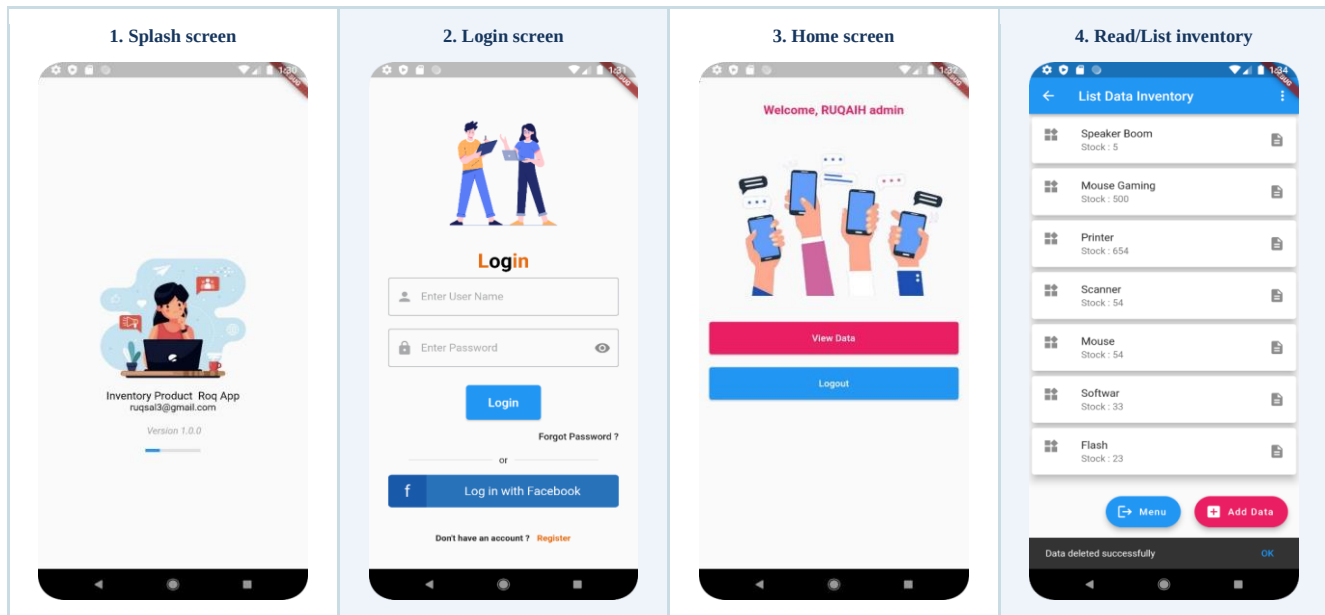
User action on screen	HTTP/API action	Visible evidence after server confirmation
Tap Add Data, enter item code/name/price/stock, then submit	POST create endpoint	Success status; return to list; fresh GET shows the new item.
Open an item, tap EDIT, change fields, submit	POST editdata.php	Success status; fresh GET/detail shows the changed values.
Tap DELETE on detail screen	POST deleteData.php	Cancel leaves the row unchanged; confirm sends DELETE.
Confirm deletion	POST deleteData.php	Fresh GET no longer contains the item; show a success message.
Login / Sign Up	POST credentials/profile to server	Server validates; passwords are hashed server-side and never returned.

Screen	Purpose in the application
SplashScreen_Roq	Application entry and branding.
LoginForm_Roq	Send credentials to the login API.
Home	Navigate to inventory operations.
List Data Inventory	GET and display inventory rows.
Add Data	POST a new inventory item.
Detail	Display the selected item; edit or delete it.
Edit Data	POST updated values.

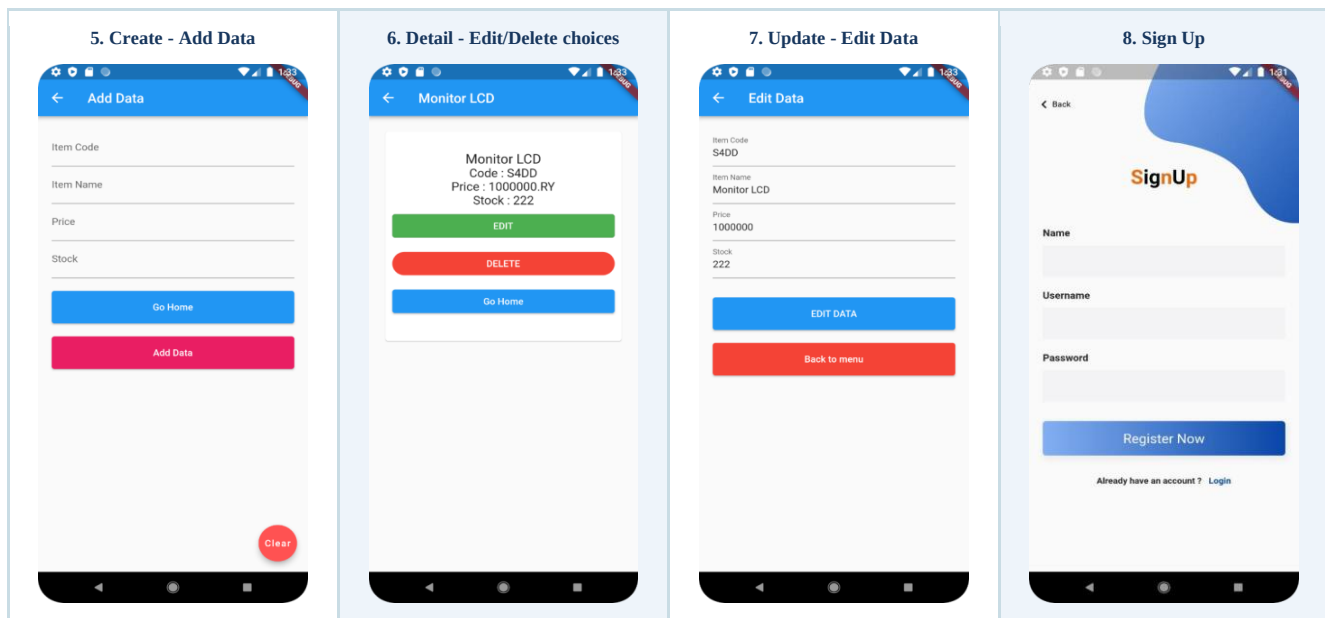
Screen	Purpose in the application
Sign Up	Create a new user account.

6.1.2 Original Application Screens

The composite workflow remains the quick overview. The following original screenshots are kept at a larger size for classroom explanation of each screen.



Original screens - application entry, login, home, and inventory list.



Original CRUD screens - create, detail/delete, update, and registration.

6.2 Local Lab Addressing: Why 10.0.2.2?

PURPOSE Reach XAMPP/PHP running on the development machine from Android Emulator.	KEY API 10.0.2.2 → host loopback	CHECKPOINT The emulator reaches the host server instead of its own localhost.
---	--	---

ANDROID EMULATOR LAB BASE URL

```
const baseUrl = 'http://10.0.2.2/my_store';  
  
// Examples:  
// $baseUrl/getdata.php  
// $baseUrl/adddata.php  
// $baseUrl/editdata.php  
// $baseUrl/deleteData.php
```

DEVICE NOTE

10.0.2.2 is specific to Android Emulator. A physical phone needs a reachable LAN/HTTPS server address. For production, use HTTPS and a real API host.

6.2.1 Original XAMPP and phpMyAdmin Setup

Use these screenshots to show the local lab environment before reading conn.php: start Apache/MySQL, open phpMyAdmin, create my_store, then confirm the tables and fields.

Start Apache and MySQL in XAMPP

XAMPP Control Panel v3.3.0 [Compiled: Apr 6th 2021]

Service	Module	PID(s)	Port(s)	Actions
Apache	7212 756	80, 443	Stop Admin Config Logs	
MySQL	4548	8080	Stop Admin Config Logs	
FileZilla			Start Admin Config Logs	
Mercury			Start Admin Config Logs	
Tomcat			Start Admin Config Logs	

Log output:
 1:54:06 AM [Apache] Attempting to start Apache app...
 1:54:06 AM [Apache] Attempting to start Apache app...
 1:54:06 AM [Apache] Status change detected: running
 1:54:06 AM [mysqld] Status change detected: running
 2:03:14 AM [mysqld] Attempting to stop MySQL app...
 2:03:14 AM [mysqld] Status change detected: stopped
 2:03:19 AM [mysqld] Attempting to start MySQL app...
 2:03:20 AM [mysqld] Status change detected: running

Open phpMyAdmin and select the database

phpMyAdmin - 4.9.1 (2021-04-01) - Database: my_store - Table: users

SELECT * FROM `users`

Recent Favorites

Structure SQL Search Insert Export Import Privileges Operations Triggers

SHOW ALL Number of rows: All Filter rows Search this table

email	password
aymeh@gmail.com	3456
aymeh@yahoo.com	3456
abu@gmail.com	3562
mada@gmail.com	45422
parag@gmail.com	23514314
row@gmail.com	45613643
ph@gmail.com	245354
mad@gmail.com	45345
parag@gmail.com	24524325
wh@gmail.com	23454345

Create the first table and fields

localhost/phpmyadmin/index.php?route=/database/structure/db/my_store

Database: my_store - Table: users

Relation view

Field	Type	Length	Flags	Comment
id	int(11)		PRIMARY, AUTO_INCREMENT	
item_code	text	2		
item_name	text	3		
price	int(11)	4		
stock	int(11)	5		

Review the inventory table structure

Database: my_store - Table: inventory

Relation view

Field	Type	Length	Flags	Comment
id	int(11)		PRIMARY, AUTO_INCREMENT	
item_code	text	2		
item_name	text	3		
price	int(11)	4		
stock	int(11)	5		

Confirm the users table fields in phpMyAdmin

Database: my_store - Table: users

Relation view

Field	Type	Length	Flags	Comment
id	int(100)		PRIMARY, AUTO_INCREMENT	
NAME	text	2		
UNAME	text	3		
UPASS	int(10)	4		

Original Flutter project folder structure

Project

- lab10_dbonline_ruqaih_salman2024
 - .dart_tool
 - .idea
 - android [lab10_dbonline_ruqaih_salman2024_android]
 - asset_roq
 - appimages
 - images
 - build
 - ios
 - jsonfile
 - productlist.json
 - lib
 - 0_json_Asset_API_baiscConcept_roq
 - 1_json_local_API_baiscConcept_roq
 - 2_api_Baisc_fetchData_onlineAPI_roq
 - 3_api_CURD_sample1_roq
 - adddata_roq.dart
 - detail_roq.dart
 - editdata_roq.dart
 - homepage_roq.dart
 - listdata_roq.dart
 - Login_roq.dart
 - main_curd_API_sampl1_roq.dart
 - signUpPage_roq.dart
 - splash_roq.dart

Original 2026 lab screenshots - XAMPP, phpMyAdmin, and project structure.

6.3 MySQL Schema

PURPOSE Create the inventory and users tables used by the application.	KEY API PRIMARY KEY + UNIQUE + numeric fields	CHECKPOINT The database fields match the Flutter forms and PHP routes.
--	---	--

MYSQL

```
CREATE DATABASE my_store
CHARACTER SET utf8mb4
COLLATE utf8mb4_unicode_ci;
USE my_store;

CREATE TABLE tb_item (
  id INT AUTO_INCREMENT PRIMARY KEY,
  item_code VARCHAR(40) NOT NULL UNIQUE,
  item_name VARCHAR(120) NOT NULL,
  price DECIMAL(10,2) NOT NULL,
  stock INT NOT NULL DEFAULT 0
);

CREATE TABLE users (
  UID INT AUTO_INCREMENT PRIMARY KEY,
  NAME VARCHAR(120) NOT NULL,
  UNAME VARCHAR(80) NOT NULL UNIQUE,
  UPASS VARCHAR(255) NOT NULL
);
```

PASSWORD HANDLING

Store a password hash in UPASS and verify the password on the server with password_verify().

6.4 PHP Connection

PURPOSE Centralize database access so every endpoint uses the same connection file.	KEY API mysqli + utf8mb4 + prepared statements	CHECKPOINT Endpoints include conn.php and use prepared statements for SQL.
---	--	--

CONN.PHP

```
<?php
$connect = new mysqli(
  'localhost',
  'root',
  '',
  'my_store'
);

if ($connect->connect_error) {
  http_response_code(500);
  exit('Database connection failed');
}

header('Content-Type: application/json; charset=utf-8');
?>
```

LOCAL LAB NOTE

The XAMPP root user with an empty password is only for local classroom practice. A deployed API must use secured database credentials.

6.4.1 connect.php - Compatibility Alias

The original project uses conn.php. Keep it as the single database connection file. If an older example calls connect.php, this short alias loads conn.php without duplicating credentials or connection logic.

php/my_store/connect.php

```
<?php
// connect.php - compatibility alias for older classroom examples
require_once __DIR__ . '/conn.php';
?>
```

WHY BOTH FILES?

conn.php is the real connection file. connect.php exists only for compatibility, so every endpoint still shares one connection implementation.

A. READ — listdata_roq.dart ↔ getdata.php

Goal: fetch all inventory rows from MySQL and display them in the Flutter list.

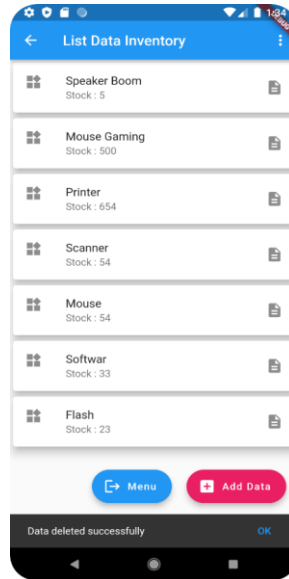


Figure 6.5 — READ result: inventory rows displayed after GET getdata.php.

READ THIS PAGE LEFT → RIGHT

listdata_roq.dart → GET → getdata.php → MySQL → UI refresh

1 Open list	2 getData()	3 GET	4 getdata.php	5 SELECT	6 ListView
----------------	----------------	----------	------------------	-------------	---------------

FLUTTER listdata_roq.dart	GET REQUEST	PHP getdata.php
1 Send GET request <pre>final response = await http.get(Uri.parse('\$baseUrl/getdata.php'),);</pre> 2 Decode server JSON <pre>final data = json.decode(response.body); return data;</pre> 3 Display with FutureBuilder <pre>FutureBuilder(future: getData(), builder: (...) => ListView.builder(...),)</pre>	GET ↓ /my_store/getdata.php SQL SELECT RESPONSE JSON array	1 Read rows from MySQL <pre>\$query = \$connect->query('SELECT * FROM tb_item ORDER BY id DESC');</pre> 2 Build a PHP array <pre>\$result = []; while (\$row = \$query->fetch_assoc()) { \$result[] = \$row; }</pre> 3 Return JSON to Flutter <pre>echo json_encode(\$result);</pre>

DATA BRIDGE — follow the same value from Flutter to MySQL

Flutter value	Request key	PHP receives	MySQL field
http.get(...)	—	No request body	tb_item rows

<code>response.body</code>	JSON array	<code>echo json_encode(\$result)</code>	<code>id, item_code, item_name, price, stock</code>
----------------------------	------------	---	---

WHAT TO UNDERSTAND

READ has no form body. Flutter asks for data; PHP selects rows and returns one JSON array.

COMPLETE FILES

`listdata_roq.dart` ↔ `getdata.php`

After the response arrives, FutureBuilder rebuilds the ListView with the returned rows.

lib/listdata_roq.dart

PAIRS WITH [getdata.php](#) • [GET](#) • [SELECT](#)

```
import 'dart:convert';
import 'package:flutter/material.dart';
import 'package:http/http.dart' as http;
import 'adddata_roq.dart';
import 'detail_roq.dart';

const baseUrl = 'http://10.0.2.2/my_store';

class Get_ListDataInventory_Roq extends StatefulWidget {
  const Get_ListDataInventory_Roq({super.key});

  @override
  State<Get_ListDataInventory_Roq> createState() =>
    _Get_ListDataInventory_RoqState();
}

class _Get_ListDataInventory_RoqState
  extends State<Get_ListDataInventory_Roq> {

  Future<List> getData() async {
    final response = await http.get(
      Uri.parse('$baseUrl/getdata.php'),
    );

    if (response.statusCode == 200) {
      return json.decode(response.body) as List;
    }
    throw Exception('Failed to load data');
  }

  void refreshList() {
    setState(() {});
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('List Data Inventory')),
      floatingActionButton: FloatingActionButton.extended(
        onPressed: () async {
          final changed = await Navigator.push(
            context,
            MaterialPageRoute(
              builder: (_) => const AddData_Roq(),
            ),
          );
          if (changed == true) refreshList();
        },
        icon: const Icon(Icons.add),
        label: const Text('Add Data'),
      ),
      body: FutureBuilder<List>(
        future: getData(),
        builder: (context, snapshot) {
          if (snapshot.hasError) {
            return Center(child: Text('${snapshot.error}'));
          }
          if (!snapshot.hasData) {
            return const Center(child: CircularProgressIndicator());
          }

          final items = snapshot.data!;
          if (items.isEmpty) {
            return const Center(child: Text('No data'));
          }

          return ListView.builder(
            itemCount: items.length,
            itemBuilder: (context, index) {
              final item = items[index];
              return Card(
                child: ListTile(
                  title: Text(item['item_name'].toString()),
                  subtitle: Text('Stock: ${item['stock']}'),
                  trailing: Text(item['price'].toString()),
                  onTap: () async {
                    final changed = await Navigator.push(
                      context,
                      MaterialPageRoute(
                        builder: (_) => Detail_Roq(
                          list: items,

```

```

        index: index,
      ),
    ),
  );
  if (changed == true) refreshList();
},
),
);
},
);
},
);
},
);
}
}
}

```

php/my_store/getdata.php

CALLLED BY `listdata_roq.dart` → `getData()` • **RETURNS JSON ARRAY**

```

<?php
require 'conn.php';

$query = $connect->query('SELECT * FROM tb_item ORDER BY id DESC');
$result = [];

while ($row = $query->fetch_assoc()) {
  $result[] = $row;
}

echo json_encode($result);
?>

```

B. CREATE — adddata_roq.dart ↔ adddata.php

Goal: take values from the Add Data form, send them with POST, and insert one new row.

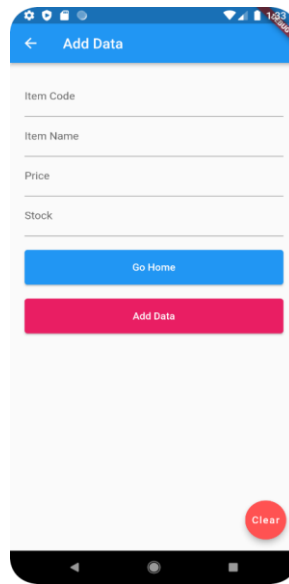


Figure 6.6 — CREATE screen: form values sent to adddata.php.

READ THIS PAGE LEFT → RIGHT

adddata_roq.dart → POST → adddata.php → MySQL → UI refresh

1 Fill form	2 addData()	3 POST	4 adddata.php	5 INSERT	6 Reload list
----------------	----------------	-----------	------------------	-------------	------------------

FLUTTER adddata_roq.dart	POST REQUEST	PHP adddata.php
1 Collect values from TextFields <pre>final body = { 'itemcode': controllerCode.text, 'itemname': controllerName.text, 'price': controllerPrice.text, 'stock': controllerStock.text, };</pre> 2 Send POST to PHP <pre>final response = await http.post(Uri.parse('\$baseUrl/adddata.php'), body: body,);</pre> 3 Continue after success <pre>if (response.statusCode == 201) { Navigator.pop(context, true); }</pre>	POST ↓ /my_store/adddata.php SQL INSERT RESPONSE 201 + JSON	1 Receive the same keys <pre>\$itemcode = \$_POST['itemcode'] ?? ''; \$itemname = \$_POST['itemname'] ?? ''; \$price = \$_POST['price'] ?? ''; \$stock = \$_POST['stock'] ?? '';</pre> 2 Insert into MySQL <pre>\$stmt = \$connect->prepare('INSERT INTO tb_item', (item_code,item_name,price,stock)', VALUES (?,?,,?)');</pre> 3 Reply to Flutter <pre>http_response_code(201); echo json_encode(['success' => true]);</pre>

DATA BRIDGE — follow the same value from Flutter to MySQL

Flutter value	Request key	PHP receives	MySQL field
controllerCode.text	itemcode	\$_POST['itemcode']	item_code
controllerName.text	itemname	\$_POST['itemname']	item_name

controllerPrice.text	price	\$_POST['price']	price
controllerStock.text	stock	\$_POST['stock']	stock

WHAT TO UNDERSTAND

The request key must match exactly on both sides: Flutter
itemcode → PHP \$_POST['itemcode'] → MySQL item_code.

COMPLETE FILES

adddata_roq.dart ↔ adddata.php

After HTTP 201, close the form and reload the list so the student sees the new row from the server.

lib/adddata_roq.dart

PAIRS WITH [adddata.php](#) • [POST](#) • [INSERT](#)

```
import 'package:flutter/material.dart';
import 'package:http/http.dart' as http;

const baseUrl = 'http://10.0.2.2/my_store';

class AddData_Roq extends StatefulWidget {
  const AddData_Roq({super.key});

  @override
  State<AddData_Roq> createState() => _AddData_RoqState();
}

class _AddData_RoqState extends State<AddData_Roq> {
  final controllerCode = TextEditingController();
  final controllerName = TextEditingController();
  final controllerPrice = TextEditingController();
  final controllerStock = TextEditingController();

  Future<void> addData() async {
    final response = await http.post(
      Uri.parse('$baseUrl/adddata.php'),
      body: {
        'itemcode': controllerCode.text,
        'itemname': controllerName.text,
        'price': controllerPrice.text,
        'stock': controllerStock.text,
      },
    );

    if (response.statusCode != 200 && response.statusCode != 201) {
      throw Exception('Add failed');
    }
  }

  @override
  void dispose() {
    controllerCode.dispose();
    controllerName.dispose();
    controllerPrice.dispose();
    controllerStock.dispose();
    super.dispose();
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('Add Data')),
      body: Padding(
        padding: const EdgeInsets.all(20),
        child: ListView(
          children: [
            TextField(
              controller: controllerCode,
              decoration: const InputDecoration(labelText: 'Item Code'),
            ),
            TextField(
              controller: controllerName,
              decoration: const InputDecoration(labelText: 'Item Name'),
            ),
            TextField(
              controller: controllerPrice,
              decoration: const InputDecoration(labelText: 'Price'),
              keyboardType: TextInputType.number,
            ),
            TextField(
              controller: controllerStock,
              decoration: const InputDecoration(labelText: 'Stock'),
              keyboardType: TextInputType.number,
            ),
            const SizedBox(height: 20),
            ElevatedButton(
              onPressed: () async {
                await addData();
                if (!mounted) return;
                Navigator.pop(context, true);
              },
              child: const Text('Add Data'),
            ),
          ],
        ),
      ),
    );
  }
}
```

```
    });  
  }  
}
```

php/my_store/adddata.php

CALLED BY `adddata_roq.dart` → `addData()` • **INSERTS ONE ROW**

```
<?php  
require 'conn.php';  
  
$itemcode = $_POST['itemcode'] ?? '';  
$itemname = $_POST['itemname'] ?? '';  
$price = $_POST['price'] ?? '';  
$stock = $_POST['stock'] ?? '';  
  
$stmt = $connect->prepare(  
    'INSERT INTO tb_item (item_code, item_name, price, stock)  
    VALUES (?, ?, ?, ?)'  
);  
$stmt->bind_param('ssdi', $itemcode, $itemname, $price, $stock);  
$stmt->execute();  
  
http_response_code(201);  
echo json_encode(['success' => true]);  
?>
```

C. UPDATE — editdata_roq.dart ↔ editdata.php

Goal: send the selected row id plus the edited values, then update that exact row in MySQL.

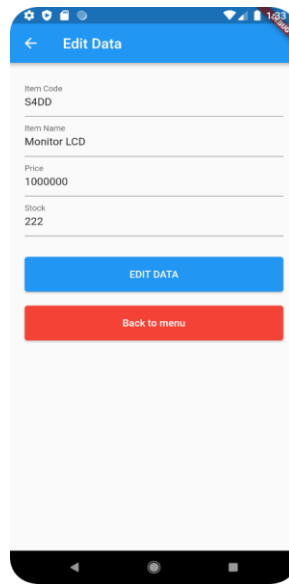


Figure 6.7 — UPDATE screen: selected record edited through editdata.php.

READ THIS PAGE LEFT → RIGHT

editdata_roq.dart → POST → editdata.php → MySQL → UI refresh

1 Open row	2 editData()	3 POST + id	4 editdata.php	5 UPDATE	6 Reload list
---------------	-----------------	----------------	-------------------	-------------	------------------

FLUTTER editdata_roq.dart	POST REQUEST	PHP editdata.php
1 Keep the selected row id <code>final id = widget.list[widget.index]['id'];</code> 2 Send id + edited values <code>final response = await http.post(Uri.parse('\$baseUrl/editdata.php'), body: { 'id': id, 'itemcode': controllerCode.text, 'itemname': controllerName.text, 'price': controllerPrice.text, 'stock': controllerStock.text, },);</code> 3 Return after success <code>if (response.statusCode == 200) { Navigator.pop(context, true); }</code>	POST ↓ <code>/my_store/editdata.php</code> SQL UPDATE RESPONSE 200 + JSON	1 Receive id and values <code>\$id = (int)(\$_POST['id'] ?? 0); \$itemcode = \$_POST['itemcode'] ?? ''; \$itemname = \$_POST['itemname'] ?? ''; \$price = \$_POST['price'] ?? ''; \$stock = \$_POST['stock'] ?? '';</code> 2 Update only this row <code>\$stmt = \$connect->prepare('UPDATE tb_item SET', 'item_code=?, item_name=?, price=?, stock=?', 'WHERE id=?');</code> 3 Reply to Flutter <code>http_response_code(200); echo json_encode(['success' => true]);</code>

DATA BRIDGE — follow the same value from Flutter to MySQL

Flutter value	Request key	PHP receives	MySQL field
<code>widget.list[index]['id']</code>	<code>id</code>	<code>\$_POST['id']</code>	<code>id</code>

controllerCode.text	itemcode	\$_POST['itemcode']	item_code
controllerName.text	itemname	\$_POST['itemname']	item_name
controllerPrice.text	price	\$_POST['price']	price
controllerStock.text	stock	\$_POST['stock']	stock

WHAT TO UNDERSTAND

UPDATE is CREATE + one extra key: id. The id tells PHP which MySQL row must change.

COMPLETE FILES

editdata_roq.dart ↔ editdata.php

After HTTP 200, return to the list and fetch again so the changed values come back from MySQL.

lib/editdata_roq.dart

PAIRS WITH [editdata.php](#) • [POST](#) • [UPDATE](#)

```
import 'package:flutter/material.dart';
import 'package:http/http.dart' as http;

const baseUrl = 'http://10.0.2.2/my_store';

class EditData_Roq extends StatefulWidget {
  final List list;
  final int index;

  const EditData_Roq({
    super.key,
    required this.list,
    required this.index,
  });

  @override
  State<EditData_Roq> createState() => _EditData_RoqState();
}

class _EditData_RoqState extends State<EditData_Roq> {
  late TextEditingController controllerCode;
  late TextEditingController controllerName;
  late TextEditingController controllerPrice;
  late TextEditingController controllerStock;

  @override
  void initState() {
    super.initState();
    final item = widget.list[widget.index];
    controllerCode = TextEditingController(text: item['item_code'].toString());
    controllerName = TextEditingController(text: item['item_name'].toString());
    controllerPrice = TextEditingController(text: item['price'].toString());
    controllerStock = TextEditingController(text: item['stock'].toString());
  }

  Future<void> editData() async {
    final response = await http.post(
      Uri.parse('$baseUrl/editdata.php'),
      body: {
        'id': widget.list[widget.index]['id'].toString(),
        'itemcode': controllerCode.text,
        'itemname': controllerName.text,
        'price': controllerPrice.text,
        'stock': controllerStock.text,
      },
    );
    if (response.statusCode != 200) {
      throw Exception('Update failed');
    }
  }

  @override
  void dispose() {
    controllerCode.dispose();
    controllerName.dispose();
    controllerPrice.dispose();
    controllerStock.dispose();
    super.dispose();
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('Edit Data')),
      body: Padding(
        padding: const EdgeInsets.all(20),
        child: ListView(
          children: [
            TextField(controller: controllerCode,
              decoration: const InputDecoration(labelText: 'Item Code')),
            TextField(controller: controllerName,
              decoration: const InputDecoration(labelText: 'Item Name')),
            TextField(controller: controllerPrice,
              decoration: const InputDecoration(labelText: 'Price')),
            TextField(controller: controllerStock,
              decoration: const InputDecoration(labelText: 'Stock')),
            const SizedBox(height: 20),
            ElevatedButton(
              onPressed: () async {
                await editData();
              },
            ),
          ],
        ),
      ),
    );
  }
}
```

```

        if (!mounted) return;
        Navigator.pop(context, true);
      },
      child: const Text('EDIT DATA'),
    ),
  ],
),
),
);
}
}

```

php/my_store/editdata.php

CALLLED BY `editdata_roq.dart` → `editData()` • **UPDATES BY id**

```

<?php
require 'conn.php';

$id = (int)($_POST['id'] ?? 0);
$itemcode = $_POST['itemcode'] ?? '';
$itemname = $_POST['itemname'] ?? '';
$price = $_POST['price'] ?? '';
$stock = $_POST['stock'] ?? '';

$stmt = $connect->prepare(
    'UPDATE tb_item
     SET item_code=?, item_name=?, price=?, stock=?
     WHERE id=?'
);
$stmt->bind_param('ssdii', $itemcode, $itemname, $price, $stock, $id);
$stmt->execute();

echo json_encode(['success' => true]);
?>

```

D. DELETE — detail_roq.dart ↔ deleteData.php

Goal: send only the selected row id, delete that row, then refresh the list.

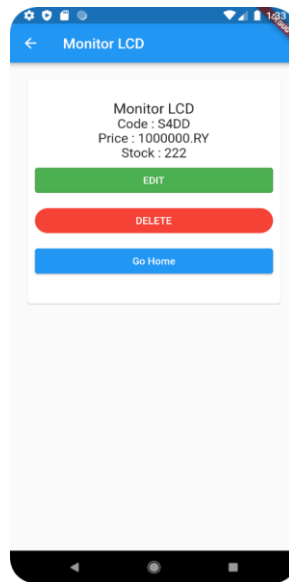


Figure 6.8 — DELETE/detail screen: selected id sent to deleteData.php.

READ THIS PAGE LEFT → RIGHT

detail_roq.dart → POST → deleteData.php → MySQL → UI refresh

1 Tap DELETE	2 deleteData()	3 POST id	4 deleteData.php	5 DELETE SQL	6 Row disappears
-----------------	-------------------	--------------	---------------------	-----------------	---------------------

FLUTTER detail_roq.dart	POST REQUEST	PHP deleteData.php
1 Read the selected id <code>final id = widget.list[widget.index]['id'];</code> 2 Send POST with id only <code>final response = await http.post(Uri.parse('\$baseUrl/deleteData.php'), body: {'id': id},);</code> 3 Leave detail after success <code>if (response.statusCode == 200) { Navigator.pop(context, true); }</code>	POST ↓ <code>/my_store/deleteData.php</code> SQL DELETE RESPONSE 200 + JSON	1 Receive id <code>\$id = (int)(\$_POST['id'] ?? 0);</code> 2 Delete by primary key <code>\$stmt = \$connect->prepare('DELETE FROM tb_item WHERE id=?');</code> <code>\$stmt->bind_param('i', \$id);</code> <code>\$stmt->execute();</code> 3 Reply to Flutter <code>http_response_code(200);</code> <code>echo json_encode(['success' => true]);</code>

DATA BRIDGE — follow the same value from Flutter to MySQL

Flutter value	Request key	PHP receives	MySQL field
<code>widget.list[index]['id']</code>	<code>id</code>	<code>\$_POST['id']</code>	<code>id</code>

WHAT TO UNDERSTAND

DELETE needs one value only: id. Flutter sends the selected id; PHP deletes WHERE id = that value.

COMPLETE FILES

detail_roq.dart ↔ deleteData.php

After HTTP 200, reload GET data. If the row is gone from the new server response, deletion is confirmed.

lib/detail_roq.dart

PAIRS WITH deleteData.php • POST • DELETE

```

import 'package:flutter/material.dart';
import 'package:http/http.dart' as http;
import 'editdata_roq.dart';

const baseUrl = 'http://10.0.2.2/my_store';

class Detail_Roq extends StatefulWidget {
  final List list;
  final int index;

  const Detail_Roq({
    super.key,
    required this.list,
    required this.index,
  });

  @override
  State<Detail_Roq> createState() => _Detail_RoqState();
}

class _Detail_RoqState extends State<Detail_Roq> {
  Future<void> deleteData() async {
    final response = await http.post(
      Uri.parse('$baseUrl/deleteData.php'),
      body: {
        'id': widget.list[widget.index]['id'].toString(),
      },
    );

    if (response.statusCode != 200) {
      throw Exception('Delete failed');
    }
  }

  Future<void> confirmDelete() async {
    final yes = await showDialog<bool>(
      context: context,
      builder: (context) => AlertDialog(
        title: const Text('Delete item?'),
        content: const Text('Are you sure?'),
        actions: [
          TextButton(
            onPressed: () => Navigator.pop(context, false),
            child: const Text('Cancel'),
          ),
          TextButton(
            onPressed: () => Navigator.pop(context, true),
            child: const Text('Delete'),
          ),
        ],
      ),
    );

    if (yes != true) return;
    await deleteData();
    if (!mounted) return;
    Navigator.pop(context, true);
  }

  @override
  Widget build(BuildContext context) {
    final item = widget.list[widget.index];

    return Scaffold(
      appBar: AppBar(title: const Text('Detail')),
      body: Padding(
        padding: const EdgeInsets.all(20),
        child: Column(
          crossAxisAlignment: CrossAxisAlignment.stretch,
          children: [
            Text('Name: ${item['item_name']}'),
            Text('Code: ${item['item_code']}'),
            Text('Price: ${item['price']}'),
            Text('Stock: ${item['stock']}'),
            const SizedBox(height: 20),
            ElevatedButton(
              onPressed: () async {
                final changed = await Navigator.push(
                  context,
                  MaterialPageRoute(
                    builder: (_) => EditData_Roq(

```

```

        list: widget.list,
        index: widget.index,
      ),
    ),
  );
  if (changed == true && mounted) {
    Navigator.pop(context, true);
  }
},
child: const Text('EDIT'),
),
ElevatedButton(
  onPressed: confirmDelete,
  child: const Text('DELETE'),
),
),
],
),
),
);
}
}

```

php/my_store/deleteData.php

CALLLED BY detail_roq.dart → deleteData() • **DELETES BY** id

```

<?php
require 'conn.php';

$id = (int)($_POST['id'] ?? 0);

$stmt = $connect->prepare('DELETE FROM tb_item WHERE id=?');
$stmt->bind_param('i', $id);
$stmt->execute();

echo json_encode(['success' => true]);
?>

```

CRUD Pairing Cheat Sheet

Use this page for revision after you understand the four operation pages.

ACTION	FLUTTER	HTTP	PHP	MYSQL
READ	getData()	GET	getdata.php	SELECT
CREATE	addData()	POST	adddata.php	INSERT
UPDATE	editData()	POST	editdata.php	UPDATE
DELETE	deleteData()	POST	deleteData.php	DELETE

1 Flutter reads UI	2 Function builds request	3 HTTP crosses boundary	4 PHP receives keys	5 SQL changes data	6 Flutter reloads
-----------------------	------------------------------	----------------------------	------------------------	-----------------------	----------------------

EXAM / LAB MEMORY TRICK

File names tell the story: listdata→getdata.php, adddata→adddata.php, editdata→editdata.php, detail/delete→deleteData.php.

6.7 Login Endpoint

PURPOSE Check the username and password sent from the login screen.	KEY API POST JSON + prepared SELECT + password_verify	CHECKPOINT The server returns loginStatus true or false.
---	---	--

USER_LOGIN.PHP

```
<?php
require 'conn.php';

$data = json_decode(file_get_contents('php://input'), true);
$username = $data['username'] ?? '';
$password = $data['password'] ?? '';

$stmt = $connect->prepare(
    'SELECT UID, NAME, UNAME, UPASS FROM users WHERE UNAME=? LIMIT 1'
);
$stmt->bind_param('s', $username);
$stmt->execute();
$user = $stmt->get_result()->fetch_assoc();

if ($user && password_verify($password, $user['UPASS'])) {
    unset($user['UPASS']);
    echo json_encode([
        'loginStatus' => true,
        'message' => 'Login Successfully',
        'userInfo' => $user
    ]);
} else {
    http_response_code(401);
    echo json_encode([
        'loginStatus' => false,
        'message' => 'Invalid Login Details'
    ]);
}
?>
```

6.7.1 Sign Up Endpoint

PURPOSE Create a new user account.	KEY API password_hash + prepared INSERT	CHECKPOINT The database stores a password hash instead of the raw password.
--	---	---

USER_REGISTER.PHP

```
<?php
require 'conn.php';

$data = json_decode(file_get_contents('php://input'), true);
$name = $data['name'] ?? '';
$username = $data['username'] ?? '';
$password = $data['password'] ?? '';

$hash = password_hash($password, PASSWORD_DEFAULT);

$stmt = $connect->prepare(
    'INSERT INTO users (NAME, UNAME, UPASS) VALUES (?, ?, ?)'
);
$stmt->bind_param('sss', $name, $username, $hash);
$stmt->execute();

echo json_encode(['success' => true]);
?>
```

PASSWORD NOTE

Registration stores a hashed password. Login checks the password with password_verify().

6.11 Remaining Complete Project Files

The complete CRUD pairs now appear beside their operation. This final block keeps the application entry and shared support files.

Use main.dart to launch the inventory list. Shared database and connection files were introduced earlier, before the CRUD operations.

HOW TO STUDY THE COMPLETE CODE

First trace the four paired functions above. Then read the complete file. Do not start by memorizing the whole page. Find the function that sends the request, the PHP file that receives it, and the database field that changes.

lib/main.dart

```
import 'package:flutter/material.dart';
import 'listdata_roq.dart';

void main() {
  runApp(const MyApp());
}

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      debugShowCheckedModeBanner: false,
      title: 'Inventory Product Roq App',
      home: const Get_ListDataInventory_Roq(),
    );
  }
}
```

OPTIONAL GITHUB / DOWNLOAD

The complete code is also supplied as a separate ZIP with the lecture. The instructor can upload the same folder to GitHub later and place the repository link or QR code in this box. The lecture remains usable even without GitHub.

7. Upload Image in Flutter and Save It in MySQL

Pick an image, upload it to PHP, and save the server path in MySQL

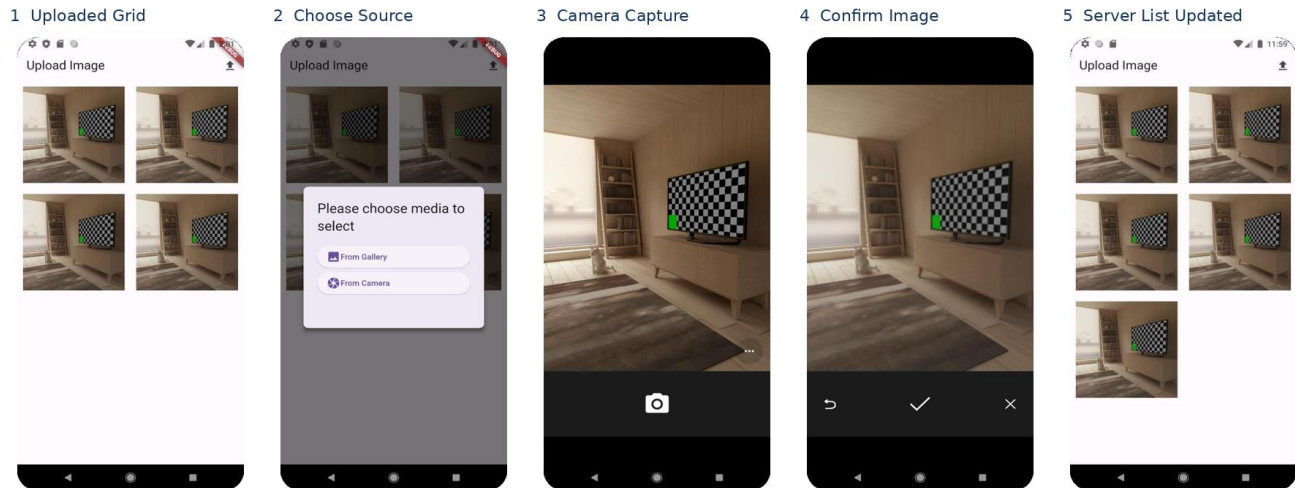
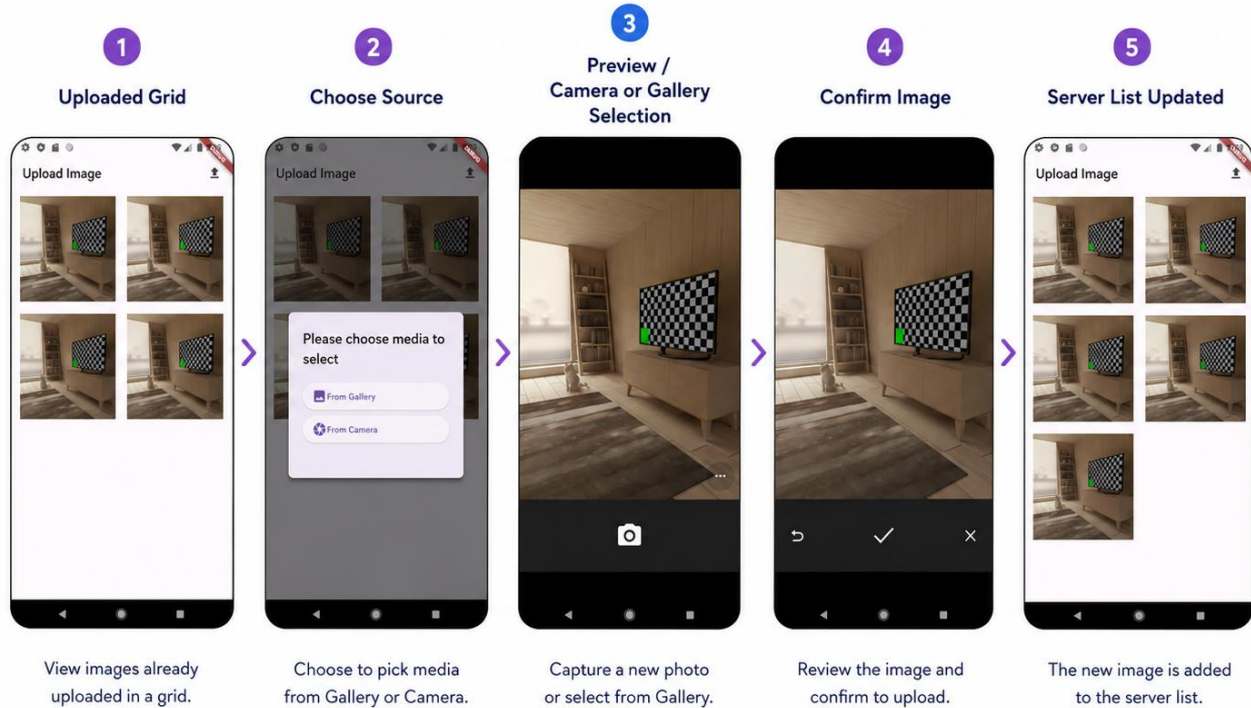


Image upload flow: image grid → choose source → camera/gallery → upload → refreshed grid.

7.1 Visual Upload Sequence

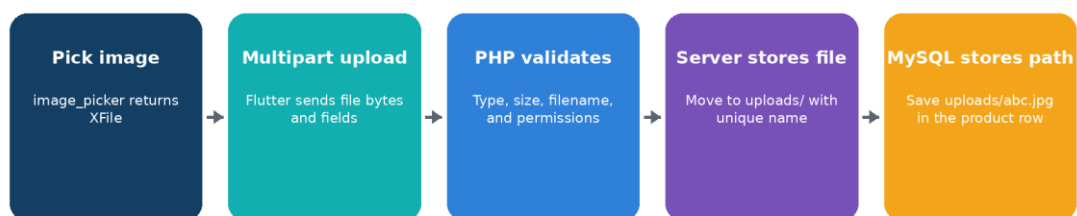
PURPOSE Follow the complete image upload flow.	KEY API choose image → multipart POST → reload	CHECKPOINT The uploaded image appears in the refreshed grid.
--	--	--

Lecture 10 Image Upload Workflow



Visual sequence — choose the source, select or capture an image, upload it, then refresh the grid.

Image upload workflow



Visual map — pick image → multipart request → PHP → server file → MySQL path.

7.2 Storage Model: Save the Server Path, Not the Phone Path

PURPOSE The phone path only exists on one device; MySQL must store a server-side relative path/URL.	KEY API image bytes → upload directory → image_path	CHECKPOINT Any client can later build the public image URL.
---	---	---

DATABASE SCHEMA

```
CREATE DATABASE practice
  CHARACTER SET utf8mb4
  COLLATE utf8mb4_unicode_ci;
USE practice;

CREATE TABLE flutter_upload_images (
  id INT AUTO_INCREMENT PRIMARY KEY,
  image_path VARCHAR(255) NOT NULL,
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

UPLOAD_CONN.PHP

```
<?php
$connect = new mysqli(
  'localhost',
  'root',
  '',
  'practice'
);

if ($connect->connect_error) {
  http_response_code(500);
  exit('Database connection failed');
}

header('Content-Type: application/json; charset=utf-8');
?>
```

WHY STORE A PATH?

The image file is stored on the server. MySQL stores its relative path so Flutter can build the image URL later.

7.3 Original Image Upload Screens

The original captures are kept below the redesigned upload map. They show the pubspec dependency and each visible phone step from choosing the image to displaying the saved result.

MySQL image-upload table: id, image, and date

MySQL image-upload table: id, image, and date

Table structure

#	Name	Type	Collation	Attributes	Null	Default	Comments	Extra	Action
1	id	int(11)			No	None		AUTO_INCREMENT	Chan
2	image	varchar(100)	latin1_swedish_ci		No	None			Chan
3	date	date			No	None			Chan

Check all With selected: Browse Change Drop Primary Unique

Print Propose table structure Track table Move columns Normalize

Add 1 column(s) after date Go

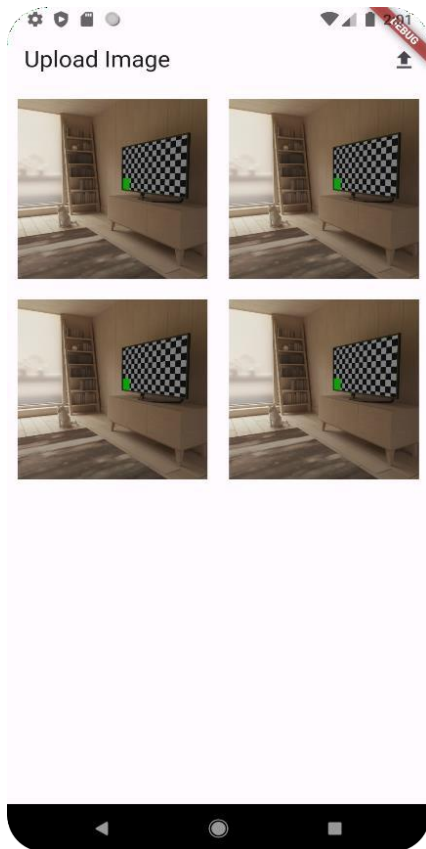
Register image_picker in pubspec.yaml

```
dependencies:
  flutter:
    sdk: flutter

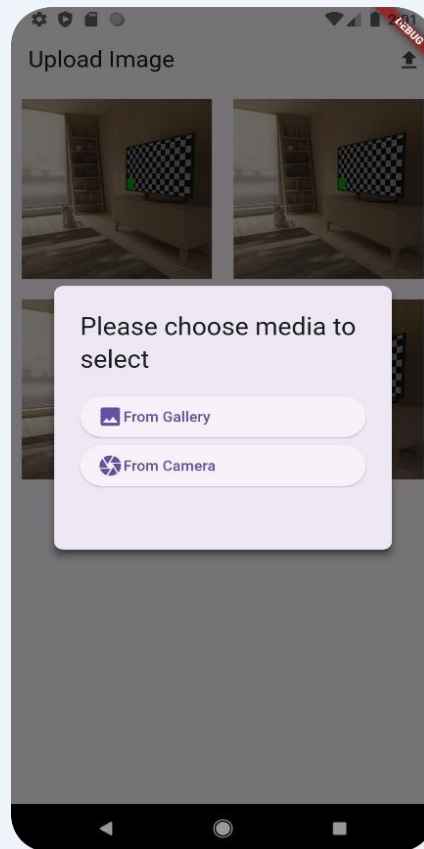
# The following adds the Cupertino Icons font to your application.
# Use with the CupertinoIcons class for iOS style icons.
cupertino_icons: ^1.0.2
image_picker:
  http:

dev_dependencies:
  flutter_test:
    sdk: flutter
```

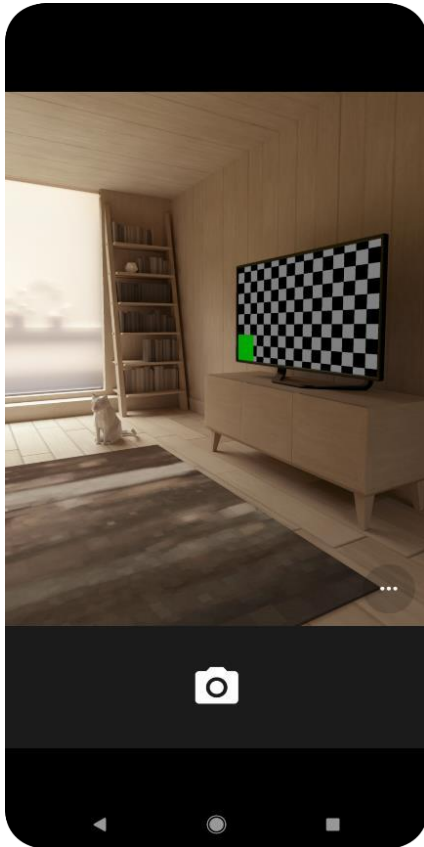
Uploaded image grid before adding a new image



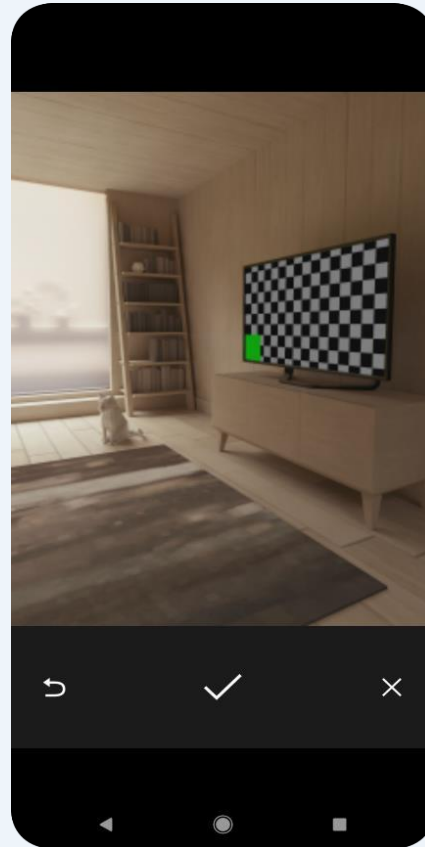
Choose Gallery or Camera



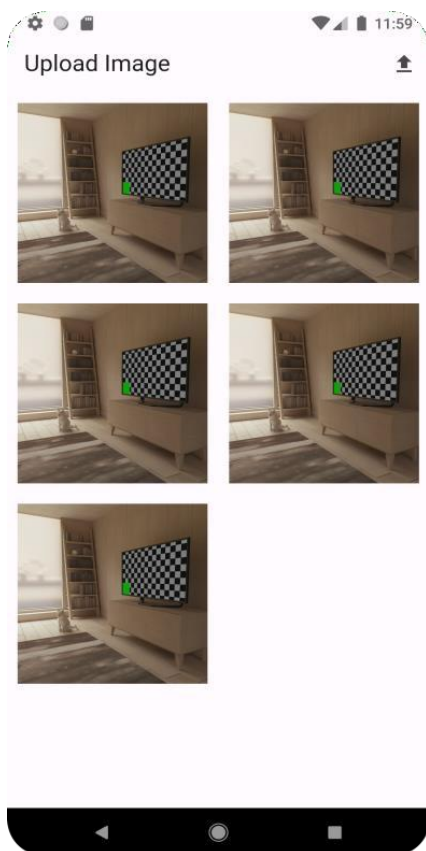
Capture or select the image



Confirm the selected image



Server image appears in the refreshed grid



Original 2026 image upload sequence retained for step-by-step demonstration.

7.4 Flutter: Pick and Upload an Image

PURPOSE Pick an image from the camera or gallery and upload it.	KEY API ImagePicker → XFile → MultipartRequest	CHECKPOINT After upload, reload the image list.
---	--	---

SETUP

```
flutter pub add image_picker http

import 'package:image_picker/image_picker.dart';
import 'package:http/http.dart' as http;
```

PICK + UPLOAD

```
final picker = ImagePicker();

Future<void> pickAndUpload(ImageSource source) async {
  final XFile? image = await picker.pickImage(
    source: source,
  );

  if (image == null) return;

  final request = http.MultipartRequest(
    'POST',
    Uri.parse('http://10.0.2.2/flutter_upload_image/create.php'),
  );

  request.files.add(
    await http.MultipartFile.fromPath('image', image.path),
  );

  final response = await request.send();

  if (response.statusCode == 200 || response.statusCode == 201) {
    getImageServer_roqFun();
  }
}
```

PLATFORM NOTE

On iOS, add the required camera and photo-library permissions in Info.plist.

7.5 PHP: Receive and Store the Image

PURPOSE Receive the uploaded image and save a server path.	KEY API \$_FILES + move_uploaded_file + INSERT	CHECKPOINT The server stores the file and returns a success response.
--	--	---

Check	Reject when	Why
Upload status	No image / upload error	Do not process an incomplete upload.
File size	Over the lab limit	Protect server resources.
MIME from temp file	Not JPEG/PNG	Do not trust filename extension.
Server filename	Never reuse client name	Avoid collisions/path tricks.
Database write	Only after file move succeeds	Do not store a path to a missing file.

CREATE.PHP

```

<?php
require 'upload_conn.php';

if (isset($_FILES['image']) || $_FILES['image']['error'] != UPLOAD_ERR_OK) {
    http_response_code(400);
    echo json_encode(['message' => 'Image is required']);
    exit;
}

if ($_FILES['image']['size'] > 5 * 1024 * 1024) {
    http_response_code(413);
    echo json_encode(['message' => 'Image is too large']);
    exit;
}

$finfo = new finfo(FILEINFO_MIME_TYPE);
$mime = $finfo->file($_FILES['image']['tmp_name']);
$allowed = ['image/jpeg' => 'jpg', 'image/png' => 'png'];
if (isset($allowed[$mime])) {
    http_response_code(415);
    echo json_encode(['message' => 'Unsupported image type']);
    exit;
}

$name = bin2hex(random_bytes(12)) . '.' . $allowed[$mime];
$relative = 'images/' . $name;
$destination = __DIR__ . '/' . $relative;
if (!move_uploaded_file($_FILES['image']['tmp_name'], $destination)) {
    http_response_code(500);
    echo json_encode(['message' => 'Could not save image']);
    exit;
}

$stmt = $pdo->prepare(
    'INSERT INTO flutter_upload_images(image_path) VALUES (?)'
);
$stmt->execute([$relative]);
http_response_code(201);
echo json_encode(['id' => (int)$pdo->lastInsertId(), 'path' => $relative]);
?>

```

UPLOAD NOTE

Check the file size and type, use a server-generated filename, and store the path only after the file is saved successfully.

8. Errors, Security, and Testing

Know the common failure cases and how to diagnose them

Failure	Example	Student-visible handling
Network / timeout	No connection, slow server	Retry; keep the entered values; show progress.
HTTP 4xx	Invalid data / unauthorized	Show validation/authentication message.
HTTP 5xx	Server/database problem	General error; detailed logging server-side.
Invalid JSON	HTML/PHP warning instead of JSON	Catch parsing failure; inspect contract.
Empty data	No rows / 204	Render intentional empty state.
Image upload rejected	Too large / wrong MIME	Show server message and keep selected image.

8.1 Basic Security Rules

Do	Do not
Use HTTPS in production	Send passwords over plain HTTP
Use prepared SQL statements	Concatenate user input into SQL
Hash passwords server-side	Store or compare plaintext passwords
Keep DB credentials on server	Place MySQL username/password in Flutter
Validate JSON fields and types	Trust the client because the UI validated

Do	Do not
Validate upload size/MIME	Trust the file extension/client filename
Return stable JSON errors	Return raw PDO/MySQL stack traces

8.2 Test the API Before the Full UI

PURPOSE Test each endpoint before connecting the full Flutter screen.	KEY API Browser/Postman + debug console + Flutter UI	CHECKPOINT You can tell whether the problem is in Flutter or in the API.
---	--	--

SERVER-SIDE VERIFICATION EXAMPLES

```
# Read
curl http://localhost/my_store/getdata.php

# Create
curl -X POST http://localhost/my_store/adddata.php \
-H "Content-Type: application/json" \
-d '{"itemcode":"M01","itemname":"Mouse","price":25.5,"stock":12}'

# Delete
curl -X DELETE "http://localhost/my_store/deleteData.php?id=1"
```

TESTING RULE

For each network example, verify the server response and then verify the matching Flutter screen.

8.3 Common Mistakes and Fast Diagnosis

Symptom	Likely cause	Check first
SocketException	Server unreachable / wrong host	Base URL, emulator 10.0.2.2, firewall.
403/401	Auth/permission failure	Headers, credentials, server policy.
FormatException	Response is not expected JSON	Print status/body; check PHP warnings.
PHP SQL error	Invalid query/column or raw input	Prepared statement + server logs.
Image uploads but does not render	Wrong server URL/path	Return/compose a reachable public URL.

9. Guided Practice Lab — Inventory API

Build the complete flow step by step

Checkpoint	Student action	Expected result
A	Run local JSON product asset	Decoded ProductItem data + shoe list screen.
B	Call RandomUser GET	HTTP 200 + user list screen.
C	Start XAMPP/PHP/MySQL	getdata.php returns a JSON array.
D	Create item	201 response + new item appears after reload.
E	Update item	Success response + changed values appear after reload.
F	Delete item	Success response + deleted item disappears after reload.
G	Login	Valid credentials succeed; invalid credentials are rejected.
H	Upload image	Image path is stored and the refreshed grid shows the image.
I	Failure path	Stop the server or send invalid input and observe the error state.

BEFORE SUBMITTING

Capture the request/result and the matching Flutter screen for each main step of the lab.

10. Knowledge Check and Assessment

Task	Prompt
Identify	Label method, endpoint, headers, body, status, and JSON in a request.
Parse	Decode a nested JSON object and read its values in Dart.
Explain	What does FutureBuilder show while a Future is loading, succeeds, or fails?
Design	Draw Flutter → PHP API → MySQL for create and read.
Code	Write simple GET, POST, update, and delete requests.
Security	Explain why MySQL credentials and plaintext passwords must never be in Flutter.
Upload	Explain why MySQL stores a server path rather than the phone file path.
Debug	Given a FormatException, list the first three things to inspect.

10.1 Quick Answer Key

- GET reads; POST creates; PUT replaces/updates all intended fields; PATCH changes selected fields; DELETE removes.
- jsonDecode converts JSON text into Dart Map/List structures that the app can read.
- FutureBuilder shows loading, data, or error while waiting for a Future.
- Flutter calls the API; the API validates and owns database credentials; MySQL remains behind the server boundary.
- The client checks the API-defined success status before trusting/parsing the response.
- Passwords are hashed and verified server-side; prepared statements bind values safely.
- Image files live on the server; the database stores a relative path or URL metadata.

11. Quick Reference and Official Sources

Current package and documentation references

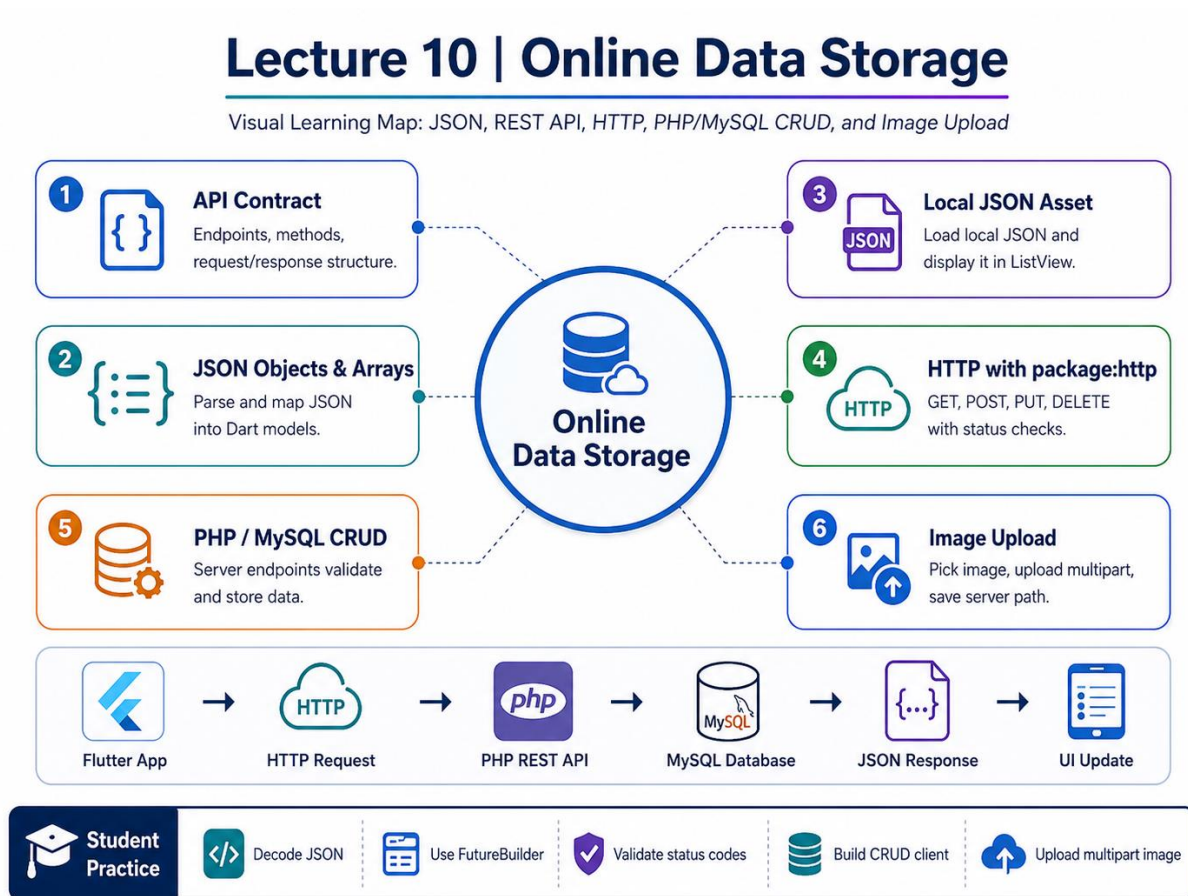
Topic	Current teaching baseline
Flutter docs	Flutter docs baseline inspected: Flutter 3.44.7; JSON guide updated 31 Jul 2026.
http package	Use flutter pub add http; current stable inspected: 1.6.0.
image_picker	Use flutter pub add image_picker; current stable inspected: 1.2.3 and XFile API.
JSON	Use dart:convert with jsonDecode for the examples in this lecture.
RandomUser	Use https://randomuser.me/api/?results=20 for the classroom GET example.
Android emulator	10.0.2.2 is the special alias for the development host loopback.
PHP/MySQL	Use server-side validation, prepared statements, and password hashing.
Version policy	Use flutter pub add to install the packages used in the project.

Official References

- Flutter JSON & serialization: <https://docs.flutter.dev/data-and-backend/serialization/json>
- Flutter networking cookbook: <https://docs.flutter.dev/cookbook/networking>
- Dart http package: <https://pub.dev/packages/http>
- Flutter image_picker: https://pub.dev/packages/image_picker
- Android Emulator networking (10.0.2.2): <https://developer.android.com/studio/run/emulator-networking-address>
- RandomUser API documentation: <https://randomuser.me/documentation>
- PHP prepared statements (mysqli): <https://www.php.net/manual/en/mysqli.quickstart.prepared-statements.php>
- PHP file upload handling: <https://www.php.net/manual/en/features.file-upload.php>

11.1 Visual Learning Map — Lecture 10 at a Glance

Use this compact map to connect the main lecture topics before the final review



Visual learning map — API contract, JSON, local assets, HTTP methods, PHP/MySQL CRUD, and image upload.